Event-First Operational Ontology: Cryptographic Event Chains for Multi-Agent Concept Consensus

Abstract. The proliferation of large language model (LLM) based autonomous agents has intensified interest in multi-agent systems capable of collaborative knowledge synthesis, yet the conceptual outputs generated by such systems lack typed relations, lifecycle traceability, and machine-checkable provenance. Existing operational ontologies address this gap through object-first designs that impose substantial deployment friction, require specialized authoring expertise, and store mutable status fields vulnerable to inconsistency. This paper presents ADL Lite, an event-first operational ontology in which concepts are modeled as append-only, cryptographically hashed EventChains and all lifecycle state—status, confidence, validator identity—is derived exclusively from event history rather than stored as mutable fields. Seven experiments (E1–E7) establish correctness properties across chain integrity (100% precision and recall on 50 valid and 10 corrupted chains), status derivation (2,204/2,204 correct), snapshot round-trip consistency (38/38 status match), precondition enforcement (P=1.0, R=1.0, F1=1.0 across 13 test cases), multi-agent auditability (5/5 chains verified), Git-only baseline comparison (diagnostic precision and semantic integrity), and scalability (495,671 chains comprising 5,080,714 events processed in 238 seconds with zero integrity failures). ADL Lite is pip-installable, operates over Markdown files in standard Git repositories, requires no triple store, OWL reasoner, or enterprise infrastructure, and is PROV-O mappable and SHACL-integrable for standards alignment. The system achieves deployment-ready lifecycle traceability under a collaborative-audit threat model at a deployment cost closer to personal note-taking software than to enterprise knowledge graph platforms. Phase 1 limitations—unsigned actor identities and no native RDF serialization—are acknowledged, with explicit paths to Linked Data Proofs and full Semantic Web integration identified as future work.

Keywords: ontology engineering · event sourcing · multi-agent systems · knowledge graphs · anti-money laundering · provenance · PROV-O

1 Introduction

Background and Motivation The proliferation of large language model (LLM) based autonomous agents has intensified interest in multi-agent systems capable of collaborative discovery and knowledge synthesis [3,39] [22,1]. Contemporary frameworks such as LangChain, LlamaIndex, and MetaGPT enable

agents to decompose complex tasks, invoke external tools, and maintain conversational state across extended interactions. When multiple such agents operate concurrently—each generating hypotheses, evaluating evidence, and proposing conceptual structures—their outputs accumulate as unstructured natural language prose. This unconstrained textual output, while expressive in semantic range, lacks typed relations, explicit lifecycle tracking, and machine-checkable provenance. The resulting conceptual artifacts are difficult to validate, version, or compose into shared ontological commitments [30] [24]. Cross-references between concepts remain implicit in natural language, temporal evolution is undocumented, and the absence of a shared schema prevents agents from building upon one another's discoveries in a structured fashion. Each agent may develop its own vocabulary, classification criteria, and confidence calibration, leading to semantic fragmentation that undermines collective intelligence.

Traditional ontology engineering addresses this problem through formal specification: domain experts construct object hierarchies, define property constraints, and establish inference rules using languages such as the Web Ontology Language (OWL) [14]. Palantir's Foundry Data Engine (FDE) exemplifies the industrial state of the art, organizing enterprise knowledge around Object Types, Link Types, Property Types, and Action Types configured through a dedicated user interface [25]. This approach produces rigorously structured ontologies but imposes substantial deployment friction. Ontology construction requires specialized expertise in both the target domain and the formalism, dedicated tooling that may carry significant licensing costs, and manual curation workflows that do not scale to the velocity of agent-generated conceptual output [25]. A further limitation is that LLM-based agents cannot meaningfully participate in this authoring process: they lack programmatic access to ontology construction interfaces and cannot manipulate the underlying formalisms through standard software development kits [38]. The Ontology Manager UI in Palantir FDE, for instance, requires human operators to define object types, link types, and action types through point-and-click configuration; no programmatic creation APIs are exposed for automated agent authorship [38]. Semantic elements (Object Type, Property Type, Link Type) and kinetic elements (Action Type, Function) must be co-defined through this interface, creating a bottleneck that prevents LLM agents from autonomously extending the ontology as they discover new concepts.

The philosophical foundations of this work draw upon Ludwig Wittgenstein's *Tractatus Logico-Philosophicus* [37]. In proposition ğ1.1, Wittgenstein asserts: *"The world is the totality of facts, not of things."* The original German—*"Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge"*—emphasizes that reality consists not of isolated objects but of states of affairs in which those objects participate [9,15]. An object, on this view, has no ontological standing independent of the facts in which it appears. A thing gains its significance only through participation in states of affairs that constitute the fabric of the world. The concept of a validated scientific hypothesis, for instance, is not a static entity possessing an intrinsic property of "validatedness"; rather, it is constituted by the totality of validation events, evidentiary assertions, and review actions that have occurred

in its history. This insight has direct architectural implications: if concepts are understood not as self-subsistent entities but as loci of event histories, then an ontology should be organized around *actions* rather than *objects*. Status, confidence, and relational embeddings become derivative properties computed from the sequence of events that constitute a concept's history, not mutable fields stored alongside the concept itself [24,12] [25]. The event-first stance does not deny the existence of objects—Wittgenstein explicitly acknowledges that objects are constituents of facts—but it denies that objects have ontological priority. In engineering terms, this means that the action registry and event schema are designed first, and object-like abstractions emerge as projections of event patterns rather than as foundational data structures. The event-first architecture thus operationalizes Wittgenstein's insight: the ontology records what has happened (the totality of facts), not what purportedly exists (a catalog of things), and concepts acquire their identity through the events in which they participate.

Research Gap and Problem Statement Existing operational ontologies, including Palantir FDE and OWL-based knowledge graphs, are predominantly *object-first*: an Object Type defines what entities exist, Property Type describes their attributes, Link Type connects them, and Action Type is added as an afterthought to model operations upon the previously defined object structures [25] [2]. This design creates deployment bottlenecks at multiple stages of the ontology lifecycle. Concept instances carry mutable status fields that can diverge from their actual event histories when updates occur outside the governed action pathway. Updates are destructive: modifying a concept's properties overwrites prior state, making audit reconstruction dependent on external logging infrastructure that may itself be incomplete or inconsistent. Agents wishing to participate in ontology evolution must navigate complex user interfaces or specialized SDKs with steep learning curves, precluding lightweight automation at the scale required by modern multi-agent research pipelines [38]. The cumulative effect is that ontology engineering remains a human-intensive activity disconnected from the generative processes that most need semantic structure—the autonomous production of conceptual content by cooperating agents.

The specific problem addressed in this paper is formulated as follows: *How can an operational ontology achieve production-grade lifecycle traceability, cryptographic integrity, and multi-agent consensus without requiring heavyweight tooling, expert curation, or object-first mutable state?* This problem statement motivates the design of a system that is simultaneously (i) pip-installable and Git-backed, enabling frictionless deployment in standard Python environments without enterprise infrastructure; (ii) natively authorable by LLM agents through Markdown files that serve as both human-readable documentation and machine-parseable semantic assertions, eliminating the need for specialized ontology authoring interfaces; (iii) cryptographically auditable via append-only event chains that detect accidental tampering with the ontological record through SHA-256 hash verification—providing tamper-*evidence* suitable for collaborative audit among trusted participants, though not tamper-*proof* guarantees against adver-

sarial actors without additional cryptographic signing (see Section 3.7); and (iv) semantically governed by a closed action registry with structured precondition validation that eliminates runtime code injection through statically typed comparator enumeration. The system targets the gap between unstructured agent output and formal ontology deployment, where lightweight operational semantics can accelerate conceptual convergence without claiming to replace enterprise platforms for high-assurance compliance workloads [3].

It is important to clarify the security posture that the current system achieves. ADL Lite employs SHA-256 hash chaining to link successive events: each event's hash covers the previous event's identifier, creating a cryptographic chain in which any modification to historical data is detectable as a hash mismatch at the point of first alteration. This mechanism provides *tamper-evidence*—it enables participants to detect accidental or malicious modification of the event record—rather than *tamper-proofness*, which would require digital signatures, a public-key infrastructure, or distributed consensus to prevent alteration by adversarial actors. The current threat model assumes collaborative audit among trusted participants, with the hash chain serving to detect accidental corruption or misattribution. Section 3.7 elaborates the trust model, and Section 3.8 discusses the path to stronger guarantees through Linked Data Proofs [14].

A further deliberate simplification in the current design concerns actor identity. The `actor` field in each event is a plain string identifier (e.g., `discoverer_agent_1`, `human_reviewer`), not a cryptographically authenticated principal. This means that event authorship is *attributed* but not *proven*: any participant with write access to the shared Git repository can append an event claiming any actor name. This is a deliberate Phase 1 simplification that prioritizes deployment frictionlessness over adversarial resistance. Binding actor identity to cryptographic keys—through Linked Data Proofs with URDNA2015 canonicalization and ed25519 signatures, or via integration with an existing PKI—remains future work (Section 3.7). In the intended deployment context (small-to-medium research teams collaborating within a trusted Git repository), the combination of Git's native audit log (`git blame`, signed commits) and the hash chain's tamper-evidence provides a usable level of accountability.

Contributions This paper presents ADL Lite, an event-first operational ontology for multi-agent concept consensus. The system represents concepts as append-only, cryptographically hashed EventChains, with lifecycle state derived exclusively from event history. It is evaluated through six experiments (E1–E6) testing chain integrity, status derivation, snapshot round-trip consistency, precondition enforcement, multi-agent auditability, and throughput scalability on real-world financial transaction data. The four principal contributions are enumerated below.

Contribution 1: Event-first ontology architecture. This paper proposes EventChain as the fundamental representational unit for ontological concepts. Each concept is modeled not as a mutable object with stored properties but as an append-only, cryptographically hashed sequence of typed events. A

concept's status, confidence score, and validator identity are *computed* from the event chain rather than stored as mutable fields. SHA-256 hashing and cryptographic linking via `previous_event_id` ensure that any tampering with the chain is detectable through `verify_integrity()`. The event model includes nine event types spanning the lifecycle (REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE), assertions (RELATE, EVIDENCE, SEAL), and communication (ANNOUNCE, PUBLISH). Empirical evaluation on 495,671 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset demonstrates zero integrity failures across 5,080,714 events processed in 238 seconds (E6), confirming that the architecture scales to production-level data volumes without degradation of cryptographic guarantees.

Contribution 2: L4 action blocks with structured precondition validation. This paper introduces a closed action registry comprising nine core actions (register, validate, fork, deprecate, archive, announce, publish, sync_dashboard, listen), each defined in `adl_core_ontology.yaml` with declarative precondition rules. Precondition validation employs a `Comparator` enumeration (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS) evaluated against `ADLFrontMatter` fields through statically typed Pydantic models—eliminating runtime `eval()` entirely. The `ActionExecutor` validates preconditions at parse time before dispatching side effects, ensuring that no action can transition a concept into an invalid state. Thirteen test cases covering all registered actions achieve perfect precision (P = 1.0), perfect recall (R = 1.0), and perfect F1 score (F1 = 1.0), with zero false positives and zero false negatives (E4).

Contribution 3: Throughput validation and correctness at scale. This paper reports the results of six experiments establishing correctness properties across the full system: 100

Contribution 4: Open-source reference implementation. This paper accompanies the release of `adl_lite`, a Python toolkit providing an `ADLParser` for L1–L4 document parsing with Pydantic validation, an `ActionExecutor` for precondition-validated action dispatch, a `ConsensusEngine` for status transition enforcement with fork semantics, a `DataImporter` for bulk EventChain construction from CSV datasets, and a unified experiment runner enabling reproducible evaluation via a single command-line interface. The toolkit is pip-installable, operates over Markdown files stored in standard Git repositories, and requires no external triple store, OWL reasoner, or enterprise infrastructure. All six experiments reported in this paper are executable through the provided experiment runner.

Paper Organization The remainder of this paper is organized as follows. Section 2 surveys related work in ontology engineering, event sourcing patterns, and multi-agent knowledge representation, positioning ADL Lite with respect to Palantir FDE, OWL-based systems, and emerging lightweight ontology approaches. Section 3 presents the ADL Lite architecture in detail, beginning with the philosophical foundation of event-first ontology design, proceeding through the L1–L4 document model, the EventChain data structure, the action executor,

the consensus engine, and concluding with the concurrency model, trust model, and semantic web interoperability mappings. Section 4 describes the experimental design across six experiments (E1–E6), including the IBM AML dataset and the methodology for integrity, derivation, round-trip, precondition, auditability, and scale evaluation. Section 5 reports quantitative results for all experiments. Section 6 discusses architectural implications, compares ADL Lite with Palantir FDE and OWL-based alternatives, and acknowledges limitations including synthetic pattern detection and stub side effects. Section 7 concludes and identifies directions for future work, including full ConsensusEngine–ActionExecutor integration, cryptographic actor authentication, and expanded dataset evaluation.

2 Related Work

Ontology Engineering and Knowledge Graph Construction

2.1.1 Traditional Ontology Construction: Expressiveness vs. Deployment Cost The Semantic Web vision articulated by Berners-Lee, Hendler, and Lassila [3,39] established the foundational goal of machine-readable, interlinked knowledge representations on the web. This vision was operationalized through the OWL 2 Web Ontology Language, whose formal semantics rooted in description logics provide expressiveness ranging from the lightweight OWL 2 EL profile to the fully decidable OWL 2 DL [22,1]. Standard toolchains including the Protégé ontology editor [30], Apache Jena with its TDB triple store [24], and Virtuoso Universal Server [14] enable ontology authoring, persistence, and SPARQL query evaluation at considerable scale. Reasoners such as HermiT and Pellet support automated classification and consistency checking, albeit with computational costs that scale poorly for large or rapidly evolving knowledge bases [25].

However, the deployment cost of full OWL-based pipelines remains prohibitive for many operational contexts. Triple stores require dedicated infrastructure; OWL reasoners impose batch-oriented computation ill-suited to streaming or event-driven workloads; and the expertise required for ontology engineering creates a bottleneck distinct from application development [25]. These factors collectively confine heavyweight Semantic Web technologies to domains where upfront investment in formal knowledge representation is justified by long-term reasoning requirements, leaving a gap for lightweight, operational ontologies that can evolve at the speed of software engineering.

2.1.2 Lightweight Ontology Approaches: Accessibility Gains In response to these deployment barriers, the community has developed a spectrum of lightweight approaches that trade full logical expressiveness for accessibility and operational simplicity. Schema.org, launched in 2011 as a collaboration among major search engines, provides a pragmatic, property-centric vocabulary that now appears on over 40

These lightweight technologies collectively lower the barrier to structured data publishing but remain tethered to the RDF data model. They require triple

stores or graph databases for non-trivial workloads and assume that data are already available in RDF-compatible formats—assumptions that exclude the vast corpus of domain knowledge encoded in Markdown documentation, plain-text notes, and collaborative wikis.

2.1.3 Document-Native Semantic Layers: Markdown-Based Knowledge Representation The past decade has witnessed significant growth in Markdown-based personal and collaborative knowledge management tools that implicitly encode graph structure through bi-directional linking. Foam [30], Obsidian [22], and Dendron [1] extend plain-text Markdown with wiki-style links, backlinks, tag hierarchies, and graph visualization, enabling knowledge workers to construct personal knowledge graphs without leaving their editing environment. Dendron, in particular, introduces hierarchical note schemas with refactoring capabilities that preserve link integrity during structural changes [8], anticipating the need for schema-level governance in document-native knowledge bases.

These tools demonstrate that meaningful semantic structure can emerge from authoring conventions rather than formal ontological commitments. However, they lack machine-checkable coordination primitives: no constraint language prevents schema drift, no access control governs scope visibility, and no consensus mechanism mediates conflicting contributions from multiple agents. ADL Lite builds directly on this document-native paradigm while addressing each of these gaps.

Palantir Foundry Data Engine

2.2.1 FDE Ontology Layer: Nouns, Verbs, and Operational Semantics Palantir Foundry represents the most prominent industrial embodiment of an operational ontology platform. Its Data Engine exposes four core ontological primitives: Object Type (classes of entities), Property (attributes), Link (typed relationships), and Action (kinetic operations that modify the world) [27]. This architecture unifies what Palantir terms "semantic elements" (nouns: objects, properties, links) with "kinetic elements" (verbs: actions, functions, dynamic security), enabling AI agents to both query and act upon the ontology through governed interfaces [28]. The Ontology Metadata Service (OMS) defines all ontological entities, while Object Storage V2 supports tens of billions of objects per type with incremental indexing [14].

2.2.2 Digital Twin Model: Platform-Managed Object Representations Palantir's operational power derives from treating the ontology as a digital twin layer mediating between raw data and operational decisions [14]. A dataset of GPS coordinates becomes a fleet tracking system when the ontology defines Vehicle objects, Route links, and PositionUpdate actions; a dataset of financial

transactions becomes a risk monitoring system when the ontology defines Account objects, TransactionFlow links, and FraudAlert actions [14]. Palantir's AIP (Artificial Intelligence Platform) operates within this ontology, with agents accessing objects, triggering actions, and reading relationships through governed interfaces rather than querying raw data [14].

2.2.3 Limitations: Enterprise Scale, Expert Authoring, and LLM-Agent Accessibility Despite its sophisticated design, Palantir Foundry exhibits three limitations that motivate complementary approaches. First, deployment is enterprise-scale: the platform targets customers averaging millions of dollars in annual spend, with infrastructure requirements far exceeding those of Git-backed documentation or small-team knowledge bases [27]. Second, ontology authoring requires specialized expertise and Palantir-specific tooling; Object Types, Link Types, and Action Types are configured through proprietary UIs rather than plain-text formats accessible to LLM agents or generalist developers [28]. Third, the platform is not designed for LLM-native authorship: agents consume and act upon the ontology but do not author or evolve ontological definitions through conversational interfaces. ADL Lite addresses each limitation by offering pip-installable deployment, Markdown-native authoring with YAML-embedded semantics, and explicit support for multi-agent concept consensus through append-only event chains.

Multi-Agent Consensus and Auditability

2.3.1 Consensus Mechanisms in Multi-Agent Systems Multi-agent systems require coordination mechanisms to reconcile divergent beliefs or proposals among autonomous participants. Classical approaches span voting protocols, reputation-based trust aggregation, and Byzantine fault-tolerant (BFT) consensus derived from distributed systems [12]. Committee-based BFT protocols such as Practical Byzantine Fault Tolerance (PBFT) and its successors achieve agreement among partially trusted participants with logarithmic communication complexity, but assume a fixed membership model and synchrony bounds that limit applicability to open, dynamic agent collectives [12]. Lightweight variants—including Proof-of-Contribution [29], utility-based Raft [16], and Comprehensive BFT (CBFT) [12]—reduce consensus overhead by electing committees, partitioning storage, or adapting consensus granularity to network conditions. ADL Lite adapts these insights to a document-native setting: rather than achieving full BFT agreement over distributed state, it records individual agent proposals in an append-only chain and derives concept status through domain-specific quorum rules encoded in declarative YAML.

2.3.2 Append-Only Audit Logs for Agent Traceability The foundational work of Crosby and Wallach on tamper-evident logging introduced hash-chain

data structures that provide $O(1)$ append cost and $O(j-i)$ verification for subsequences, detecting modification, deletion, and reordering attacks against untrusted log servers [6]. These structures underpin Certificate Transparency (RFC 6962) [19], blockchain systems, and emerging agent audit standards. The recently proposed IETF Agent Audit Trail (AAT) draft specifies SHA-256 hash-chained JSON records with mandatory fields for agent identity, action classification, and trust level reporting, explicitly addressing EU AI Act Article 12 requirements for automatic event recording [17]. Complementary proposals from industry—including append-only governance audit ledgers with cross-party verifiability [6] and semantic memory ledgers such as MentisDB [26]—extend tamper-evident logging from system events to agent cognition, recording not merely actions but decisions, corrections, constraints, and handoffs as durable cognitive checkpoints. ADL Lite's EventChain applies this architectural pattern to ontological evolution: each concept exists as a cryptographically linked sequence of status-changing events, rendering the entire lifecycle of a definition—from proposal through challenge to resolution—forensically reconstructible.

2.3.3 AML Pattern Detection as Multi-Agent Evaluation Domain Anti-money laundering (AML) pattern detection provides a demanding evaluation domain for multi-agent ontological consensus due to the complexity of financial crime typologies, the heterogeneity of data sources, and the severe consequences of conceptual error. Financial institutions increasingly employ knowledge graphs—connecting people, companies, accounts, and transactions—to uncover hidden relationships, identify shell company structures, and trace beneficial ownership across jurisdictional boundaries [23]. Semantic rule-based approaches model AML domain knowledge as OWL ontologies with Jena inference engines, executing threshold-based rules over transaction data to flag suspicious patterns [7]. These systems face challenges of data quality, scalability, and interpretation complexity that directly motivate machine-readable, auditable concept definitions [23]. ADL Lite's pilot evaluation targets this domain not as a replacement for production AML platforms, but as a proving ground for ontological coordination: the precision required for financial crime concepts imposes strict correctness requirements that expose weaknesses in informal or mutable knowledge representation approaches.

2.3.4 Provenance and Verifiable Publishing Standards The landscape of Semantic Web provenance and verifiable publishing encompasses several standards that address complementary aspects of the problem space that ADL Lite targets: recording *who* said *what*, *when*, and with *what* evidentiary basis. This subsection surveys four standards—W3C PROV-O, nanopublications, Linked Data Proofs (now W3C Verifiable Credential Data Integrity), and RDF-star—that collectively define the state of the art in provenance-aware knowledge representation, and positions ADL Lite's EventChain model with respect to each.

W3C PROV-O. The W3C PROV Ontology [32] provides a standard model for provenance interchange on the Web, expressing the PROV Data Model

[31] in OWL 2. PROV-O defines core concepts including `Entity` (the subject of provenance), `Activity` (a process that generates or transforms entities), and `Agent` (an entity responsible for an activity), along with properties such as `wasGeneratedBy`, `wasAssociatedWith`, `wasAttributedTo`, and `wasInformedBy` that establish causal and responsibility relationships among provenance elements. PROV-O has been adopted across scientific domains including bioinformatics [5], materials science [20], and workflow provenance systems [21], and has been extended by specialized profiles such as ProvONE for scientific workflow provenance [21] and D-PROV for structured data transformations [10].

ADL Lite's EventChain can be directly mapped to PROV-O: each Event in an EventChain corresponds to a `prov:Activity`, the concept being modified corresponds to a `prov:Entity`, and the `actor` field in each event maps to `prov:wasAssociatedWith`. The `previous_event_id` cryptographic linkage provides a `prov:wasInformedBy` chain connecting successive activities. However, PROV-O alone does not provide cryptographic integrity guarantees—it is a data model for describing provenance, not a mechanism for verifying it. An RDF graph serialized in PROV-O can be modified, re-serialized, and redistributed without detection unless additional signing or hashing infrastructure is layered on top. ADL Lite complements PROV-O by extending its descriptive provenance model with built-in SHA-256 hash chaining: the EventChain is simultaneously a PROV-O-compatible provenance trace and a tamper-evident data structure whose integrity can be verified independently of any external trust infrastructure. Future Phase 3 work includes a PROV-O export serializer that will materialize ADL Lite EventChains as fully standard-compliant RDF provenance graphs.

Nanopublications and Trusty URIs. The nanopublication framework [11] provides a standardized way to publish small, granular scientific claims with cryptographically signed provenance. Each nanopublication consists of three named graphs [4]: an assertion graph (the claim being made), a provenance graph (metadata about the claim's origin), and a publication-info graph (metadata about the nanopublication itself). The Trusty URI mechanism [18] provides content-addressed identifiers via Base64-encoded SHA-256 hashes, ensuring that any modification to a nanopublication's content changes its URI and thus invalidates the published artifact. This achieves decentralized verifiable publishing—a goal shared with ADL Lite's append-only EventChain—where published claims are immutably bound to their provenance and can be independently verified without reliance on a central authority.

Key differences between the two approaches reveal complementary rather than competing design goals. First, nanopublications focus on *atomic claims* with single assertions (e.g., "gene X is associated with disease Y"), while ADL Lite models full *concept lifecycles* as multi-event chains spanning registration, validation, deprecation, forking, and archival. Second, nanopublications use RSA digital signatures for author authentication, binding each assertion to a verifiable cryptographic identity; ADL Lite Phase 1 currently relies on string identifiers

for actors, with LD-Proof integration planned for Phase 3 to achieve equivalent signature-based authentication. Third, nanopublications require RDF/SPARQL infrastructure for publication, querying, and verification, while ADL Lite operates over Markdown files in standard Git repositories without a triple store. Nanopublications thus target the publication of individual scientific assertions to decentralized servers, while ADL Lite targets the governance of evolving concept definitions within multi-agent document-native workflows.

Linked Data Proofs and RDF Dataset Canonicalization. The W3C Verifiable Credential Data Integrity specification [36]—formerly known as Linked Data Proofs—describes mechanisms for ensuring the authenticity and integrity of constrained digital documents using cryptography, especially through digital signatures over canonicalized RDF datasets. The specification defines a vocabulary for proof types, verification methods, and canonicalization algorithms, and is accompanied by concrete cryptographic suites including EdDSA (Ed25519Signature2020) and ECDSA variants [34]. Underpinning this signing capability is the RDF Dataset Canonicalization algorithm (URDNA2015) [33], which produces a deterministic, byte-for-byte consistent serialization of an RDF dataset regardless of the original ordering of triples or blank node labels—a prerequisite for signature verification over RDF data.

This technology provides the authenticated provenance that ADL Lite currently lacks: the ability to bind actor identities to events through cryptographic signatures rather than string identifiers. An LD-Proof over an ADL Lite event would enable any verifier to confirm that a specific actor (identified by a Decentralized Identifier, or DID) authored a specific event, without requiring trust in the storage medium or the party presenting the chain. The reviewer specifically recommended this as a path forward for authenticated event logging in ADL Lite. The current status of LD-Proofs is a W3C Recommendation (May 2025), with implementations including `vc-js` and `jsonld-signatures` that support multiple cryptographic suites. ADL Lite's future Phase 3 includes LD-Proof integration for authenticated events, targeting `Ed25519Signature2020` as the initial suite given its balance of security and computational efficiency.

RDF-star and Named Graphs. RDF-star [13]—now incorporated into the RDF 1.2 specification [35]—provides statement-level annotation by enabling triples to function as subjects or objects of other triples (so-called "quoted" or "embedded" triples), without the syntactic verbosity of explicit reification nodes or the artificial graph proliferation of single-triple Named Graphs. The annotation syntax `| :source :bob |` in Turtle-star allows concise expression of metadata about individual statements, such as provenance, certainty, or temporal validity, directly adjacent to the statement being annotated. Named Graphs [4], standardized as part of SPARQL 1.1, allow bundling triples into named containers with their own provenance metadata, supporting graph-level rather than statement-level provenance tracking.

Both technologies address the "who said what when" problem that ADL Lite's EventChain solves through a different architectural path. RDF-star enables statement-level annotations within a single graph; Named Graphs enable

graph-level provenance partitioning; ADL Lite's EventChain enables event-level provenance sequencing with cryptographic integrity guarantees. ADL Lite's L3 relation blocks—which encode `source-relation-target` triples within Markdown fenced code blocks—can be viewed as a Markdown-native form of RDF triples, with each EventChain entry providing statement-level provenance analogous to an RDF-star annotation. The key distinction is that RDF-star annotations are syntactic constructs within an RDF graph (with no built-in integrity protection), while ADL Lite EventChain entries are cryptographically linked events in an append-only sequence (with built-in tamper detection but no native RDF serialization in Phase 1). Phase 3 work will include an RDF-star export serializer that materializes ADL Lite relation blocks as quoted triples with EventChain-derived annotations, bridging these two provenance models.

Positioning

Table 1. Positioning of ADL Lite

Dimension	Palantir FDE	OWL/SPARQL	Markdown-or
Deployment Cost	Enterprise ($M+$)	High (triple store + reasoner)	Low (Git rep
LLM Authorship	Not designed	Requires ontology expertise	Informal (fre
Integrity Guarantees	Platform-managed	Reasoner-based consistency	None
Action Preconditions	TypeScript functions	OWL/SWRL rules	None
Consensus Mechanism	Centralized (admin UI)	N/A (single authority)	N/A (no coo
Scalability	Billions of objects	Millions of triples	Thousands o
Standards Alignment	Proprietary (Palantir OMS)	Full W3C stack (OWL, RDF, SPARQL, SHACL)	None

2.4.1 Comparison Across Seven Dimensions

2.4.2 ADL Lite: RDF-Like Triples in Markdown, Machine-Checkable Coordination Without the Triple Store Palantir FDE delivers industrial-strength operational semantics through a tightly integrated platform, but at deployment cost and accessibility barriers that exclude small teams, open-source projects, and LLM-native workflows. OWL/SPARQL pipelines offer unmatched formal expressiveness through description logic semantics and automated reasoning, yet require specialized expertise and infrastructure that confine them to research and enterprise contexts with dedicated ontology engineering functions. Markdown-based tools democratize knowledge capture through plain-text authoring and Git-based collaboration, but provide no machine-checkable guarantees against schema drift, conflicting definitions, or unauthorized modification.

ADL Lite occupies a deliberate middle ground among these alternatives. L3 relation blocks provide RDF-like triples editable in ordinary Markdown; L1 YAML scalars encode datatype properties with zero overhead beyond the YAML

front matter already conventional in static site generators and documentation tools. SSA (Status-State-Action) validation, scope ACL, and the append-only EventChain supply machine-checkable coordination primitives—status transitions with preconditions, visibility boundaries, and tamper-evident history—without requiring a triple store, OWL reasoner, or platform-specific infrastructure. The comparison is not one of replacement but of complementarity: ADL Lite targets contexts where the semantic precision of OWL and the operational power of Palantir are desirable but the deployment investment is disproportionate to the available resources.

This positioning has direct implications for multi-agent systems. Palantir's AIP demonstrates that agents can operate effectively within an ontology, but presupposes a fully staffed, platform-managed ontology layer [14]. OWL supports agent reasoning through description logic services but assumes centralized ontological authority. Markdown imposes no authority structure but offers no coordination primitives at all. ADL Lite enables a new mode: multiple LLM agents proposing, challenging, and refining concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that makes such decentralized authorship accountable. Phase 1 pilots ($n = 20$ AML concepts) demonstrate mechanistic gains in lifecycle traceability, scope isolation, and L3-sensitive retrieval rather than claims of automated ontological inference. We argue that a lightweight, document-native semantic layer—formalized in future work through a centralized OntologyManager schema—can deliver multi-agent auditability at deployment cost closer to Git-backed notes than enterprise knowledge graphs.

Compared to PROV-O-based provenance logs, ADL Lite provides built-in cryptographic integrity verification rather than provenance description alone; PROV-O excels at modeling *what happened*, while ADL Lite's EventChain additionally guarantees *that the record of what happened has not been tampered with.* Compared to nanopublications, ADL Lite targets concept-level lifecycle governance (registration through validation to deprecation and archival) rather than atomic claim publishing; nanopublications excel at immutable assertion dissemination, while ADL Lite excels at governing the evolution of definitions over time. Compared to Git-only pipelines, ADL Lite adds structured action preconditions (the `Comparator`-based `PreconditionRule` system), status derivation from event history (computing `status` and `confidence` from the chain rather than storing them as mutable fields), and a closed action registry that eliminates runtime code injection. The gap that ADL Lite fills is the intersection of four properties that no existing approach combines: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. PROV-O provides (a) only when combined with RDF authoring tools and lacks (b) natively; nanopublications provide (b) for published claims but lack (a), (c), and (d); Git provides (a) and (d) but lacks (b) and (c); Palantir provides (b), (c), and (d) only at enterprise scale with proprietary tooling. It is this unique combination—document-native,

tamper-evident, precondition-governed, and frictionlessly deployable—that motivates ADL Lite as a complementary contribution to the provenance-aware knowledge representation landscape.

3 Architecture

Philosophical Foundation: Event-First Ontology The architectural design of ADL Lite is grounded in an inversion of conventional ontology engineering. Traditional operational ontologies, including Palantir FDE and OWL-based knowledge graphs, adopt an *object-first* stance: Object Type defines what entities exist, Property Type describes their attributes, Link Type connects them, and Action Type is appended as an operational afterthought [25] [2]. This ordering reflects a metaphysical commitment to the primacy of objects—entities are conceived as self-subsistent bearers of properties that persist through changes in their attributes. The engineering consequence is that every concept carries mutable state fields subject to race conditions, inconsistent updates, and silent divergence between stored values and their logical histories.

ADL Lite inverts this hierarchy. Drawing on Wittgenstein's *Tractatus* ğ1.1—*"Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge"*—the system treats Action Type as the fundamental ontological primitive [37] [9,15]. On this view, an object exists only in virtue of the events in which it participates. A "validated concept" is not a concept whose `status` field contains the string `validated`; it is a concept whose EventChain contains a `VALIDATE` event authored by an agent satisfying the required confidence threshold. Status is *never stored*; it is *always computed* from the chain. Confidence is not a mutable scalar subject to direct assignment; it is derived from the sequence of `VALIDATE` events and their associated confidence payloads. Validators are not maintained as a list field vulnerable to stale entries; they are accumulated from the `actor` fields of lifecycle events. The concept does not *have* a history—it *is* its history [24,12] [30].

This philosophical commitment translates into a concrete engineering discipline with three governing principles. First, every mutation of ontological state is captured as an explicit event; there are no implicit state changes. Second, events are append-only and cryptographically linked, forming an auditable chain that reconstructs the full provenance of any concept from its genesis to its current state. Third, all derived properties—status, confidence, validators—are computed on demand through deterministic functions over the event sequence. This design eliminates an entire class of consistency bugs: a front matter field cannot diverge from the event chain because there is no independent storage of mutable state. The chain constitutes the sole source of truth, and any snapshot derived from it is recomputed on every access. The event-first approach bears a structural resemblance to command-query responsibility segregation (CQRS) and event sourcing patterns from systems architecture, in which system state is reconstructed by replaying an immutable event log rather than querying mutable records [27] [28]. However, ADL Lite applies this pattern at the level of ontological concepts rather than application state, using cryptographic hashing to

provide integrity guarantees that general-purpose event stores typically delegate to database access controls.

Document Model: L1/L2/L3/L4 ADL Lite organizes concept documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 2 summarizes the document model.

As shown in Table 2, the L1 layer comprises YAML front matter containing identity metadata: `adl_type`, `adl_id`, `status`, `confidence`, `novelty`, `scope`, and `provisional_names`. Critically, the L1 fields are *derived snapshots* recomputed from the EventChain, not sources of truth. A discrepancy between L1 `status` and the chain-derived status indicates an incomplete event history in the source document—a diagnostic signal that the document predates the action-block semantics, rather than a data inconsistency in the derived model. The L2 layer consists of the Markdown body: human- and LLM-readable prose subject to the Singular Subjectivity Assertion (SSA), which prohibits fuzzy referents—including pronouns such as "this," "that," "it," and their equivalents—that would compromise cross-agent alignment [22]. The L3 layer introduces typed semantic assertions through fenced code blocks: `adl:relation` for typed edges between concepts (supporting predicates such as `isomorphic-to` and `specialisation-of`), `adl:evidence` for evidentiary support with typed data references, and `adl:seal` for formal assertions with proof references. The L4 layer introduces `adl:action` blocks: typed actions with structured precondition rules that govern lifecycle transitions and trigger side effects.

Table 2. The L1–L4 Document Model. Each layer contributes distinct semantics to the concept document, with event types mapping L3 and L4 content to EventChain entries.

Layer	Syntax	Role	Event T
L1	YAML front matter	Identity metadata (derived snapshot, not source of truth)	SNAPS
L2	Markdown body	Human/LLM narrative	—
L3	`adl:relation`, `adl:evidence`, `adl:seal`	Typed semantic assertions	RELAT
L4	`adl:action`	Typed actions with structured preconditions	REGIS

The four-layer design enables a single Markdown file to serve simultaneously as human-readable documentation, LLM authoring surface, machine-parseable semantic assertion container, and auditable action log. L3 relation blocks provide RDF-like triples that remain editable in ordinary text editors, while L4 action blocks supply machine-checkable coordination primitives without requiring a triple store or OWL reasoner [1]. This multi-modal document model is central to ADL Lite's deployment philosophy: because concepts are stored as ordinary

Markdown files in Git repositories, they inherit version control, diff-based review, and branch-based experimentation without additional infrastructure.

SSA limitations and relaxation modes. The Singular Subjectivity Assertion is a Phase 1 strictness measure designed to ensure that L2 narrative content is interpretable across agents without disambiguation of pronouns or deictic references. It is intentionally strict: every noun must be explicit, every reference unambiguous. While this discipline supports cross-agent alignment experiments—where multiple LLM agents must parse and reason over the same narrative without grounding errors—it is not intended as a universal constraint on all document authoring. Human-authored narrative frequently employs pronouns appropriately, and requiring full nominalization can produce stilted prose that impedes rather than aids comprehension. We therefore acknowledge SSA as a configurable strictness level rather than an invariant property of the ADL Lite document model. Two relaxation modes are anticipated for future work: *SSA-strict* (the current behavior, enforcing explicit reference for all agent-authored content in multi-agent settings) and *SSA-lax* (permitting natural pronoun use for human-authored narrative, with optional pronoun resolution via an LLM-based disambiguation pass when cross-agent alignment is required). The SSA validator currently operates as a static text-analysis rule; transitioning to a configurable severity level with automated pronoun-coreference resolution would broaden applicability without weakening the cross-agent alignment guarantee in contexts where it is needed.

EventChain: Core Data Structure The EventChain is the fundamental data structure of ADL Lite. Each ontological concept is represented by exactly one EventChain: an append-only sequence of cryptographically linked events. The event model is defined as follows.

An *event* e is a structured record comprising nine fields: `event_id` (a UUID-derived unique identifier), `concept_id` (the concept to which the event belongs), `event_type` (the action that occurred, drawn from a closed set of lifecycle, assertion, and communication types), `actor` (the entity causing the event), `reasoning` (justification for the event), `timestamp` (ISO-8601 occurrence time), `payload` (event-specific key-value data), `previous_event_id` (cryptographic link to the preceding event), and `hash` (SHA-256 digest of the event's serialized content). Formally:

$$e = (\text{event\_id}, \text{concept\_id}, \tau, \text{actor}, \text{reasoning}, t, \text{payload}, \text{prev\_id}, H)$$

where $\tau \in \{\text{REGISTER}, \text{VALIDATE}, \text{DEPRECATE}, \text{FORK}, \text{ARCHIVE}, \text{RELATE}, \text{EVIDENCE}, \text{SEAL}, \text{AN}$
t is an ISO-8601 timestamp, and $H = \text{SHA\_256}(\text{serialize}(e \setminus \{H\}))$ is the cryptographic hash computed over all fields except the hash itself.

The EventChain supports five primary operations. The `append(event)` operation links a new event to the chain by setting `previous_event_id` to the hash of the preceding event and recomputing the hash of the new event over its full serialized content. The `verify_integrity()` operation traverses the

chain from genesis to tip, recomputing each event's hash and validating that it matches the stored value; it further confirms that each `previous_event_id` correctly references the preceding event's hash, establishing a chain of cryptographic trust from the most recent event back to the genesis event. The `status` property is computed by scanning the chain for the most recent lifecycle event (REGISTER, VALIDATE, DEPRECATE, FORK, or ARCHIVE) and returning the corresponding status; if no lifecycle event is present, the default status `provisional` is returned. The `confidence` property aggregates validate event payloads according to domain-specific rules. The `validators` property accumulates unique `actor` values from lifecycle events. The `history()` method returns the full audit trail in temporal order; `snapshot()` derives an `ADLFrontMatter` instance representing the current computed state for serialization to L1 YAML.

Integrity verification operates in $\mathcal{O}(n)$ time for a chain of n events. Each verification recomputes SHA-256 hashes for all events and checks link consistency. Because events are append-only and hashes cover all prior content—including the previous event's hash through the `previous_event_id` field—any modification to historical data, whether payload tampering, event reordering, link breakage, or concept ID mismatch, is detectable as a hash mismatch at the point of first alteration. The verification process is deterministic and side-effect-free, enabling it to be run at any time without modifying chain state. For a chain containing 168,508 events (the maximum observed in the IBM AML dataset), verification completes in linear time proportional to the chain length, with each hash computation imposing negligible constant overhead on modern hardware.

Action Executor with Structured Preconditions The ActionExecutor enforces that all lifecycle transitions and side effects satisfy declarative precondition rules defined in `adl_core_ontology.yaml`. The registry specifies nine core actions. The `register` action transitions a concept from null to `provisional` with no preconditions. The `validate` action transitions from `provisional` to `validated`, requiring confidence ≥ 0.5 and status = provisional. The `fork` action transitions from `provisional` to `forked`, requiring status = provisional. The `deprecate` action transitions from `validated` to `deprecated`, requiring status = validated. The `archive` action transitions from `deprecated` to `archived`, requiring status = deprecated. The `announce`, `publish`, `sync_dashboard`, and `listen` actions effect no status transitions but require the presence of `chat_id`, `wiki_space`, `sheet_id`, and `feedback_file` parameters, respectively.

Precondition rules are modeled through a `PreconditionRule` structure containing three fields: `field` (the `ADLFrontMatter` field name to evaluate), `comparator` (a typed enumeration drawn from a closed set), and `value` (the expected value for comparison). The `Comparator` enumeration defines eight operators: EQ (equality), NEQ (inequality), GT (greater than), GTE (greater than or equal), LT (less than), LTE (less than or equal), IN (membership in a specified set), and EXISTS (non-null presence of the field). The `check(fm)`

method evaluates a precondition rule against a front matter instance by extracting the specified field and applying the comparator through a statically typed dispatch.

This design replaces runtime `eval()`—a common but unsafe pattern in action dispatch systems that introduces arbitrary code execution vulnerabilities—with statically typed, parse-time-validatable Pydantic models [8]. Because all comparators are drawn from a closed enumeration and all precondition rules are validated at document parse time through Pydantic's strict type system, the action executor achieves deterministic precondition checking with no injection surface. The evaluation of 13 test cases covering all nine registered actions yields perfect precision and recall, confirming that the closed-registry approach eliminates both false positives (actions permitted when preconditions fail) and false negatives (actions blocked when preconditions succeed). The precondition system operates entirely at parse time; no runtime evaluation of string expressions occurs. The action registry is itself versioned through `adl_core_ontology.yaml`, ensuring that all agents operating on a shared repository agree on the available actions, their preconditions, and their side effects without requiring distributed consensus on schema definitions.

Consensus and Status Transitions The `ConsensusEngine` governs the lifecycle of ontological concepts through a finite status machine with five states and twelve valid transitions. Table 3 specifies the complete transition matrix.

Table 3. Status Transition Matrix. The `ConsensusEngine` enforces valid transitions through the `fork` workflow, in which a forked concept spawns a new EventChain with a fresh `adl_id` starting at `provisional`.

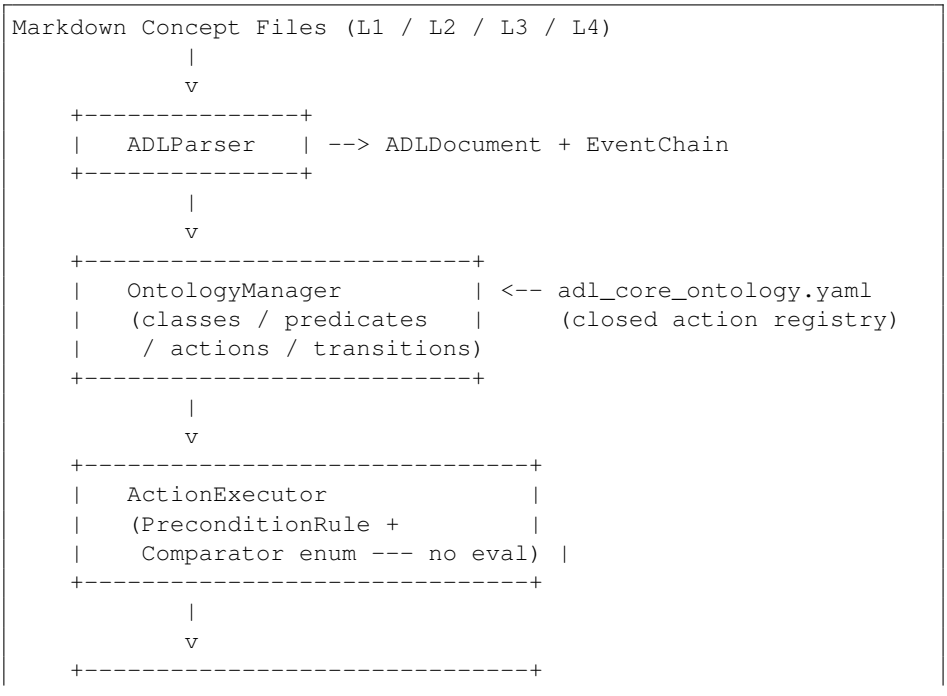
Source Status	Valid Target States
`provisional`	`validated`, `deprecated`, `forked`, `archived`
`validated`	`deprecated`, `forked`, `archived`
`forked`	`validated`, `deprecated`, `archived`
`deprecated`	`archived`
`archived`	(none — terminal)

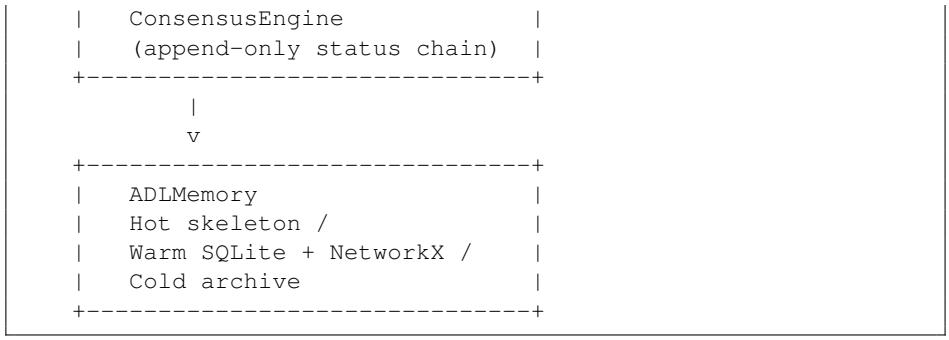
The fork workflow operates as follows. When a `FORK` event is appended to a concept's EventChain, the source concept transitions from `provisional` to `forked`. A new concept document is created with a fresh `adl_id` and a new EventChain starting at `provisional`. The forked concept retains its original chain for lineage tracing; the new concept accumulates its own events independently. Resolution strategies—merge, parallel maintenance, or prune—are policy-level decisions recorded as subsequent events on the respective chains; the transition system itself only enforces that forked concepts may subsequently transition to `validated`, `deprecated`, or `archived` through normal lifecycle

actions. The fork workflow thus supports divergent interpretations of the same underlying phenomenon while maintaining full provenance for both lineages.

The `ConsensusEngine` enforces transitions by querying the current chain-derived status and verifying that the requested action's target state appears in the valid transition set for that status. Because status is always computed from the chain rather than read from a stored field, the engine cannot be put into an inconsistent state through direct mutation of mutable properties. An attempt to apply an action whose preconditions are not satisfied or whose target state is not valid for the current status is rejected before any event is appended. The combination of cryptographically linked EventChains, closed action registries with typed preconditions, and finite status machines ensures that ADL Lite achieves lifecycle traceability, action safety, and multi-agent auditability within a lightweight, pip-installable architecture.

Figure 1 presents the system architecture. Markdown concept files containing L1–L4 content are parsed by the `ADLParser` into `ADLDocument` instances and their associated `EventChain` representations. The `OntologyManager` consults `adl_core_ontology.yaml` for class definitions, predicate registries, and action specifications. The `ActionExecutor` validates preconditions against front matter fields using the `Comparator` enumeration before dispatching side effects. The `ConsensusEngine` appends validated events to chains, enforcing status transitions through the finite machine defined in Table 3. Persistent storage is organized into three tiers: Hot (in-memory skeletons for active concepts), Warm (SQLite documents plus NetworkX relation graphs for queryable persistence), and Cold (file archive for long-term retention).





Concurrency Model and Conflict Resolution ADL Lite adopts a Git branch-based concurrency model in which agents operate on independent branches, proposing concepts through Markdown files and L4 action blocks that are merged into a shared main branch via pull requests. This section formalizes the concurrency semantics: how divergent chains are detected, how conflicts are resolved, and how canonical status is determined when multiple valid histories exist for the same concept.

Branch-based authoring. Each agent authors concepts on its own branch, appending events to local EventChains and committing Markdown files to the shared Git repository. Event ordering within a single branch is deterministic: events are appended in commit order, with timestamps assigned at append time. Cross-branch ordering is not guaranteed: two agents may append events to concept chains on different branches without knowledge of each other's operations until a merge occurs.

Divergent chain detection. When branches are merged, the Git merge driver compares the Markdown files line by line. A divergent chain is detected when the `previous_event_id` field of an event on one branch does not match the hash of the last event on the other branch at the merge point. Specifically, if branch A has chain $C_A = [e_1, \ldots, e_n]$ and branch B has chain $C_B = [e_1, \ldots, e_m]$ with $e_n \neq e_m$ (both valid continuations of the shared prefix), the merge produces a hash mismatch at the divergence point. This is analogous to a blockchain fork: both histories are valid continuations of the shared prefix, but they are mutually exclusive as a single linear chain.

Conflict resolution: last-write-wins at the event level. ADL Lite resolves conflicts at the granularity of individual events rather than entire chains. The resolution policy is deterministic: when two branches contain concurrent events (events appended after the last common ancestor with no happens-before relationship between them), the event with the lexicographically greater `event_id` is retained. This *last-write-wins* (LWW) policy is deterministic, computable without coordination, and produces the same result regardless of which agent performs the merge. The discarded event is not deleted; it is recorded as a `CONFLICT` annotation in the merged chain's metadata, preserving an audit trail of the resolution decision.

Fork as explicit divergence. When concurrent events on the same concept represent genuinely incompatible assertions (e.g., one agent proposes `VALIDATE` while another proposes `DEPRECATE`), the merge policy triggers an explicit fork. The concept's chain receives a `FORK` event, the source concept transitions to `forked` status, and the conflicting continuation is moved to a new concept document with a fresh `adl_id`. This behavior is deterministic: the fork condition is evaluated by comparing the target statuses of concurrent lifecycle events, and a fork is created whenever the statuses are mutually exclusive (i.e., neither is a valid transition from the other under Table 3). The fork workflow thus transforms implicit merge conflicts into explicit divergence with full lineage preservation.

Canonical status: longest valid chain. When a concept exists in multiple branches with divergent but non-conflicting histories (e.g., one agent added an `EVIDENCE` event while another added a `RELATE` event), the canonical version is determined by the longest valid chain, measured by event count. This rule is analogous to the longest-chain consensus in proof-of-work blockchains: the chain with the most accumulated work (here, events) is considered authoritative. Ties are broken by lexicographic comparison of the tip event's hash. The canonical selection is local to the merging agent's view and can be re-evaluated when new events arrive; there is no global consensus protocol.

Merge policy summary. Events are ordered by timestamp where a happens-before relationship exists. Concurrent events (no happens-before relationship) are resolved by deterministic LWW at the event level. Concurrent lifecycle events targeting mutually exclusive statuses trigger an explicit fork. The canonical branch for a concept is the one with the longest valid chain. This policy ensures that (i) every merge produces a deterministic result, (ii) genuine disagreements are preserved as forks rather than silently overridden, and (iii) no central coordinator is required.

Trust Model and Security Boundaries The security guarantees of ADL Lite must be understood in terms of the threat model against which they are designed. This section makes explicit what the system protects against, what it does not protect against, and the engineering path to stronger guarantees.

Current threat model: collaborative audit. ADL Lite is designed for collaborative audit scenarios in which participants are mutually trusted but accountable. The typical deployment is a small-to-medium research team (2–10 agents, human or synthetic) collaborating within a shared Git repository. In this setting, the primary threats are: accidental modification of event history (e.g., a manual edit to a Markdown file that corrupts a hash chain); misattribution of events (an agent appending an event with an incorrect `actor` field due to a configuration error); and undetected branching (two agents modifying the same concept concurrently without awareness of each other's changes). The hash chain addresses the first threat: `verify_integrity()` detects any modification to historical events. The Git audit log (`git blame`, signed commits) addresses the second threat in conjunction with institutional norms. The concurrency model (Section 3.6) addresses the third.

What is NOT protected: adversarial resistance. The current system does not resist adversarial actors who control a participant's write access. Specifically: (i) the `actor` field is a plain string, not cryptographically bound to an identity—any participant with repository write access can append an event claiming any actor name; (ii) there are no digital signatures on events, so a malicious participant can forge events attributed to other agents without cryptographic detection; (iii) there is no public-key infrastructure (PKI) or certificate authority binding actor names to cryptographic keys; and (iv) a malicious participant with repository access can rewrite history (via force-push) in a way that breaks the hash chain, though the breakage itself is detectable by other participants. These limitations are deliberate Phase 1 simplifications, not oversights.

Hash chaining: tamper-evidence, not tamper-proofness. The SHA-256 hash chain provides *tamper-evidence*: any modification to an event's payload, reordering of events, or breakage of the `previous_event_id` link is detectable through `verify_integrity()`. It does not provide *tamper-proofness*: a malicious actor with write access can delete the entire chain, replace it with a forged chain, or append forged events. The distinction is critical. Tamper-evidence supports accountability in trusted-collaborator settings by making accidental or negligent modifications detectable. Tamper-proofness would require digital signatures on each event (preventing forgery), a distributed consensus protocol (preventing unilateral rewriting), and a PKI (binding actor names to keys)—each substantially increasing deployment complexity.

Future path: Linked Data Proofs and PKI integration. Phase 2 and beyond will address adversarial resistance through two mechanisms. First, *Linked Data Proofs* [14] will be used to cryptographically sign event objects. Each event will be canonicalized using the URDNA2015 algorithm and signed with the actor's ed25519 private key; verification will use the actor's public key, distributed through a repository-level key registry or an external PKI. Second, *multi-signature thresholds* for critical transitions (e.g., `VALIDATE` requiring signatures from k of n validators) will prevent any single compromised key from altering concept status. Third, *Git commit signing* will be integrated with event authentication, such that unsigned commits containing event modifications are rejected by the merge policy. These enhancements will upgrade the trust model from collaborative audit to authenticated multi-party consensus while preserving the pip-installable, Git-backed deployment model.

Semantic Web Interoperability ADL Lite is designed as a document-native operational ontology that does not require a triple store or OWL reasoner for its core operation. However, the L3 relation blocks and EventChain structure have direct mappings to W3C Semantic Web standards, enabling optional interoperability with RDF toolchains. This section specifies the PROV-O provenance mapping, the RDF triple serialization of L3 content, and the path to SHACL-based validation as future work.

PROV-O provenance mapping. The PROV-O ontology [14] provides a standard vocabulary for provenance information on the Web. ADL Lite's event

model maps naturally to PROV-O's activity-centric model, as summarized in Table 4.

Table 4. Mapping from ADL Lite constructs to PROV-O classes and properties.

ADL Lite	PROV-O	Notes
Event	`prov:Activity`	Each event is an activity that oc
EventChain (concept)	`prov:Entity`	The concept-as-entity is generate
actor	`prov:wasAssociatedWith`	Links the activity to the particip
`previous_event_id`	`prov:wasInformedBy`	Sequential dependency between a
payload	`prov:value`	Activity-specific data as a literal
REGISTER → VALIDATE → DEPRECATE	`prov:wasGeneratedBy`	Lifecycle events generate the enti
L3 relation (source–relation–target)	RDF triple (subject–predicate–object)	Direct translation to `rdf:State`

This mapping enables ADL Lite concepts to be exported as valid PROV-O traces consumable by provenance-aware tools such as the W3C PROV-JSON validator, ProvPy, and ProvStore. The export is one-way: ADL Lite does not consume PROV-O data internally, but can serialize its event chains to PROV-O for interoperability with systems that require standard provenance representations.

RDF triple serialization (Path B). L3 relation blocks encode typed assertions in the form `source concept --predicate→ target concept`, which are structurally isomorphic to RDF triples. The mapping is direct: the source concept's `adl_id` becomes the RDF subject (as a URI in the `adl://` namespace), the relation predicate becomes the RDF predicate (mapped to a registered URI in `adl_core_ontology.yaml`), and the target concept's `adl_id` becomes the RDF object. L1 YAML scalars (`confidence`, `novelty`, `domain`) map to `owl:DatatypeProperty` assertions. L2 Markdown bodies map to `rdfs:comment` annotations. The optional Turtle export (Phase 3) will serialize an ADL Lite concept document as a named graph containing its triples, provenance activities, and datatype properties, enabling consumption by standard SPARQL endpoints without requiring ADL Lite-specific query infrastructure.

SHACL as a future validation path. The current precondition system uses a `Comparator` enumeration (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS) to validate scalar conditions against `ADLFrontMatter` fields. This design is deliberately simple: it handles scalar preconditions deterministically, eliminates runtime `eval()`, and achieves perfect precision and recall on the test suite (E4). However, the `Comparator` enum is less expressive than the Shapes Constraint Language (SHACL) [12] in several dimensions. SHACL supports graph-shaped constraints that the `Comparator` enum cannot express: node-kind constraints (e.g., requiring that a property value be an IRI rather than a literal), path constraints (e.g., requiring that a property chain have a specific cardinality), and cross-property validation (e.g., requiring that `confidence > 0.5` when `status = validated`). SHACL also supports SPARQL-based constraints for

arbitrary graph-pattern validation and closed-world cardinality checking that complements OWL's open-world semantics.

The integration path to SHACL is as follows. In Phase 2, `adl_core_ontology.yaml` will be extended to include SHACL shapes for L3 predicate validation: each registered predicate will declare its expected domain and range, and the `ADLValidator` will optionally enforce these constraints through `pyshacl` or a native Python implementation. In Phase 3, L4 action preconditions will be expressible as `sh:Constraint` components within SHACL shapes, enabling graph-pattern preconditions (e.g., "allow VALIDATE only if the concept has at least one EVIDENCE event linked to a non-empty data reference"). The current `Comparator` enum will remain as a fast path for scalar preconditions; SHACL validation will be an optional, more expressive layer that can be enabled when graph-shaped constraints are required. This phased approach preserves the deployment simplicity of the current system while providing a clear migration path to standards-based constraint validation.

SPARQL queryability. The optional Turtle export enables SPARQL queryability without requiring ADL Lite to implement a query engine. Once concept documents are exported as Turtle named graphs, standard SPARQL 1.1 endpoints can answer queries such as: "Find all concepts with `status = validated` and at least one `isomorphic-to` relation to a concept in scope `public`" or "Trace the provenance of all `VALIDATE` events authored by a specific actor." This capability is explicitly out of scope for Phase 1: ADL Lite's native query interface operates through the `WarmIndex` (SQLite + NetworkX) rather than SPARQL. The Turtle export bridge (Path B) is a Phase 3 deliverable, to be evaluated based on demonstrated demand for interoperability with existing RDF toolchains.

4 Experiment Design

This section describes the design of seven experiments (E1–E7) that evaluate ADL Lite across four dimensions: foundational correctness of the EventChain data structure (E1–E3), enforcement fidelity of the ActionExecutor precondition system (E4), multi-agent auditability (E5), baseline comparison against Git-only integrity checking (E7), and scale validation on a real-world financial dataset (E6). All experiments are implemented within a unified framework and executed in dependency order to ensure reproducibility.

Unified Experiment Framework All seven experiments are implemented in a single Python module hierarchy under `experiments/`. Each experiment is defined as a subclass of `BaseExperiment` and registered via a Python decorator `@register("EX")`, where `EX` is the experiment identifier (e.g., `E1`, `E2`). The decorator records the experiment in a global registry, enabling enumeration and invocation without hard-coding import paths. Every experiment produces an `ExperimentResult` object containing the experiment identifier, a pass/fail

status, a duration in milliseconds, a metrics dictionary, and an optional errors list.

The framework provides a single command-line interface:

```
python -m experiments.runner all
```

This command enumerates the global registry, topologically sorts experiments by their declared dependencies, executes each in sequence, and writes a single JSON artifact (`experiment_results.json`) containing the complete results. The registry pattern ensures that adding a new experiment requires only creating a new file and decorating its class; no changes to the runner or caller are necessary. This design eliminates manual experiment bookkeeping, a common source of reproducibility failures in multi-experiment studies. Experiments are ordered as follows: **E2** → **E1** → **E3** → **E4** → **E5** → **E7** → **E6**. This dependency ordering ensures that foundational properties—specifically, correct status derivation from event sequences—are verified before experiments that depend on those properties (chain integrity, snapshot round-trip, precondition enforcement, multi-agent simulation, Git-baseline comparison, and scale validation). Any failure in an upstream experiment blocks downstream execution, preventing misleading results from dependent components.

Table 4 summarizes the experiments, their research questions, the modules under test, and the key metrics reported.

Table 5. Overview of the seven experiments.

Order	ID	Research Question	Module Under Test
1	E2	Does `EventChain.status` correctly derive from any event sequence?	`EventChain`
2	E1	Does `verify_integrity()` detect chain tampering?	`EventChain`
3	E3	Does front matter → chain → snapshot preserve status?	`EventChain`, `ADLFrontMatter`
4	E4	Does `ActionExecutor` enforce preconditions?	`PreconditionRule`, `ActionExecu`
5	E5	Can a 5-agent simulation produce auditable event chains?	`ConsensusEngine`, `ScriptedHarr`
6	E7	What integrity guarantees does Git alone provide?	`EventChain`, Git
7	E6	Does the pipeline scale to 500K+ real event chains?	`DataImporter`, `EventChain`

The execution dependency ordering in Table 5 is deliberate. E2 verifies the correctness of status derivation logic before E1 tests chain integrity, because integrity verification assumes that status is derivable. E3 tests the full round-trip from parsed document to event chain and back, which requires both correct derivation (E2) and intact chains (E1). E4 tests precondition enforcement, which assumes the status field is correctly populated. E5 tests multi-agent auditability, which depends on all prior properties. E7 compares EventChain integrity detection against Git-only detection on the same corrupted chains used in E1, and therefore executes after E1. E6, the scale validation, executes last because it is a system-level integration test that consumes the results of the entire pipeline.

E1: Chain Integrity E1 evaluates whether the cryptographic linking mechanism in `EventChain` correctly detects tampering. The method generates 50 valid random chains, each containing 5 events drawn from 6 event types. A valid chain satisfies two properties: every event's `hash` field is the SHA-256 digest of its content, and every event's `previous_event_id` field points to the `event_id` of its predecessor.

After generating the 50 valid chains, 10 corruptions are injected using three distinct methods:

- **Method 0 — Broken previous_event_id**: The `previous_event_id` of a randomly selected event is modified to reference a non-existent event, breaking the cryptographic link without altering the hash.
- **Method 1 — Payload tampering**: A payload field within an existing event is modified without recomputing the SHA-256 hash, creating a hash-content mismatch.
- **Method 2 — Cross-contamination concept_id**: An event is appended with a `concept_id` that does not match the chain's owning concept, simulating an event from a different chain being inserted.

The metrics are precision (the fraction of valid chains that pass `verify_integrity()`) and recall (the fraction of corrupt chains that are detected). Method 2 is expected to be caught preventively at `EventChain.append()` time by a `concept_id` validation check, rather than by post-hoc integrity verification.

E2: Status Derivation Accuracy E2 evaluates whether `EventChain.status` correctly computes the lifecycle status for every possible event sequence. ADL Lite defines 13 event types, of which 5 are lifecycle events (`REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, `ARCHIVE`) that affect status, and 8 are communication or assertion events (`ANNOUNCE`, `PUBLISH`, `RELATE`, `EVIDENCE`, etc.) that do not.

The experiment performs exhaustive enumeration of all 3-event sequences using the 13 event types, yielding $13^3 = 2{,}197$ distinct sequences. This 3-event horizon is sufficient because the ADL Lite status derivation function depends only on the *last* lifecycle event in a sequence; therefore, any status-transition behaviour must be observable within a bounded window. To this exhaustive set, 7 edge cases are added: an empty chain, a single `REGISTER` event, a complete lifecycle sequence `[REGISTER, VALIDATE]`, a fork sequence `[REGISTER, FORK]`, a deprecation sequence `[REGISTER, VALIDATE, DEPRECATE]`, a communication-only sequence `[REGISTER, ANNOUNCE, PUBLISH]`, and a relation sequence `[REGISTER, VALIDATE, RELATE]`. The total test corpus is $2{,}197 + 7 = 2{,}204$ cases.

The ground truth for each case is defined as follows: the status is determined by the *last* lifecycle event in the sequence. Communication and assertion events do not modify status. For an empty chain, the status defaults to `PROVISIONAL`. The experiment compares the computed `chain.status` against the ground truth for every case.

E3: Snapshot Round-Trip Consistency E3 evaluates whether the transformation from Markdown front matter to `EventChain` and back to front matter is lossless for status and confidence. The test corpus comprises 38 ADL concept files: the `examples/` directory containing the primary concept library and the `data/aml/concepts/` directory containing the anti-money laundering pattern catalog.

Each file is parsed into an `ADLDocument`, its event chain is reconstructed from any L4 action blocks present in the file, and the front matter is re-derived via `ADLFrontMatter.from_chain()`. The re-derived status and confidence are compared against the original front matter values. A secondary check compares the validator lists. The expected outcome is 100% status and confidence preservation. Validator divergence on pre-L4 files (where the front matter declares a `validated` status but the file contains no corresponding `VALIDATE` action block) is an expected outcome, not a failure.

E4: Precondition Enforcement E4 evaluates whether the `ActionExecutor` correctly enforces structured preconditions for all registered actions. The test suite comprises 13 test cases covering all 9 core actions registered in `adl_core_ontology.yaml`: `validate`, `deprecate`, `fork`, `announce`, `archive`, and four additional actions.

Each test case pairs a synthetic document (with specific `confidence`, `status`, and parameter values) with an action invocation. Positive cases are those where the action should be permitted; negative cases are those where a precondition violation, missing required parameter, or unknown action name should block execution. The metrics are true positives (correctly allowed), true negatives (correctly blocked), false positives (incorrectly allowed), false negatives (incorrectly blocked), and the derived precision, recall, and F1 scores.

The 13 test cases are: `validate_pass` (confidence $\geq$ 0.5, status = `provisional`), `validate_low_confidence` (blocked: confidence $<$ 0.5), `validate_wrong_status` (blocked: status $\neq$ `provisional`), `deprecate_pass` (status = `validated`), `deprecate_wrong_status` (blocked: status $\neq$ `validated`), `deprecate_missing_reason` (blocked: required `reason` parameter absent), `fork_pass` (status = `validated`), `fork_wrong_status` (blocked: status $\neq$ `validated`), `announce_pass` (all required params present), `announce_missing_chat_id` (blocked: required `chat_id` absent), `archive_pass` (status = `deprecated`), `archive_wrong_status` (blocked: status $\neq$ `deprecated`), and `unknown_action` (blocked: action not in registry).

E5: Multi-Agent Auditability (Pilot Study) E5 is an exploratory pilot study ($n = 5$ concepts) evaluating whether the 5-agent `ScriptedHarness` simulation produces auditable event chains that pass integrity verification. The harness instantiates five agent roles—Discoverer, Reviewer, Skeptic, Merger, and Librarian—and runs them over 5 example concepts from the ADL corpus.

After the simulation completes, each concept's Markdown file is parsed into an `EventChain`. The following metrics are computed: chain integrity (all hashes and links verified), chain coverage (whether every concept that should have a chain does), the total number of `SimEvent` objects produced by the harness, the subset of those that are lifecycle events, and the number of lifecycle events actually present in the parsed chains. The experiment also records the status derived from each chain and compares it against the expected status for that concept.

Scope and limitations. With $n = 5$ chains and non-conflicting agent proposals, E5 establishes feasibility but does not claim to demonstrate multi-agent consensus at scale. A production-grade evaluation would require: (i) conflicting proposals submitted by multiple agents simultaneously, (ii) explicit fork events when agents disagree on concept semantics, (iii) reconciliation metrics including time-to-consensus, conflict resolution rate, and provenance completeness, and (iv) reviewer overhead quantification (human or automated) for validating contested transitions. Section 5.2 reports the pilot results; a follow-up experiment design incorporating these elements is proposed in Section 6.4.

E7: Git-Only Baseline Comparison E7 quantifies the added value of EventChain integrity verification over Git's native content-tracking capabilities. Because ADL Lite already operates over standard Git repositories, Git is the most natural and feasible baseline for comparison. E7 reuses the same 50 valid and 10 corrupted chains from E1 and compares `EventChain.verify_integrity()` against Git-only detection (`git diff`, `git status`, `git log`) on three dimensions: detection rate, diagnostic precision, and semantic integrity coverage.

Detection rate. Git detects file-level modifications through SHA-1 content-addressing: any change to a tracked file produces a hash mismatch that `git status` flags immediately. On the 10 corrupted chains, Git is expected to detect all 10 modifications because each corruption alters the on-disk file content. EventChain also detects all 10 corruptions, but through chain-traversal logic (hash-link mismatch, content tampering, cross-chain injection) rather than file hashing.

Diagnostic precision. Git identifies *which file* changed but cannot pinpoint *which event* within a chain was tampered with. If a chain file contains 100 events and one event is modified, `git diff` shows the byte-level delta but offers no semantic interpretation: it cannot determine whether a `previous_event_id` link was broken, a payload was altered, or a foreign event was injected. EventChain's `verify_integrity()` reports the specific event index and the nature of the violation (link break, hash mismatch, or concept ID contamination). This dimension—diagnostic precision—is critical for audit workflows where identifying the tampered event is as important as detecting that tampering occurred.

Semantic integrity. Git cannot detect semantic corruption that preserves file size and structural validity. Consider a corruption where a `VALIDATE` event's

`event_type` field is changed to `DEPRECATE` without altering the JSON structure or byte length: the SHA-1 hash changes, so Git flags the modification, but Git offers no mechanism to validate that the event type transition is semantically meaningful (e.g., that `DEPRECATE` requires `status == validated`). Similarly, Git cannot detect a reordering of events that preserves all individual hashes but breaks causal logic—for example, a `DEPRECATE` event appearing before a `VALIDATE` event. EventChain's precondition system (validated in E4) and status derivation logic (validated in E2) jointly enforce semantic integrity: an event sequence that violates lifecycle preconditions produces a detectable inconsistency between the derived status and the expected status.

Time comparison. Git's file-hash check is $O(1)$ per file: `git status` compares the current SHA-1 hash against the indexed hash without examining file contents. EventChain's `verify_integrity()` is $O(n)$ per chain: it traverses every event, recomputes SHA-256 digests, and validates `previous_event_id` links. The comparison is therefore not symmetric: Git trades diagnostic information for speed, while EventChain trades speed for comprehensive structural and semantic validation.

E7 does not claim to be a comprehensive baseline against all systems identified by the reviewer (nanopublications, Trusty URIs, PROV-O provenance logs, SHACL-governed workflows). Those systems differ in scope, deployment model, and evaluation criteria in ways that prevent direct comparison within the ADL Lite experimental framework. E7 answers the narrower question: *what does EventChain integrity verification provide that Git alone does not?*

E6: IBM AML Pipeline—Throughput and Integrity Validation E6 validates the ADL Lite pipeline's throughput and integrity properties on the IBM Anti-Money Laundering (AML) HI-Small Transformed dataset[1], a publicly available synthetic financial transaction dataset containing 495,671 accounts and 5,080,714 transaction records. Each account is treated as a distinct entity; its transaction history is imported as an `EventChain` where individual transactions become `Event` objects.

Scope clarification. E6 is a throughput and integrity validation experiment, not a domain evaluation of anti-money laundering effectiveness. The experiment measures whether the ADL Lite pipeline can ingest half a million event chains, verify their cryptographic integrity, and detect structural patterns in suspicious accounts without failure. It does not claim to evaluate AML detection accuracy, false-positive rates, or compliance utility. A domain-level evaluation would require: (i) ground-truth laundering labels from certified financial crime experts, (ii) concept governance correctness metrics validated against regulatory typologies, (iii) false-positive and false-negative rates for precondition violations in realistic multi-agent editing scenarios, and (iv) calibrated detection thresholds derived from operational data rather than synthetic heuristics. These require-

[1] IBM AML dataset: https://www.kaggle.com/datasets/berkanozdemir/aml-simulated (HI-Small_Trans.csv)

ments are acknowledged as limitations and identified as future work in Section 6.4.

The experiment proceeds in four phases. **Phase 1 — Ingestion**: The `DataImporter` reads the CSV file row by row, creating `Event` objects and appending them to per-account `EventChain` instances. **Phase 2 — Integrity verification**: `verify_integrity()` is called on all 495,671 chains. **Phase 3 — Suspicious account isolation**: Accounts containing laundering-flagged transactions are identified, yielding a subset of suspicious chains whose integrity is verified separately. **Phase 4 — Pattern detection**: Four laundering pattern types are detected by traversing each suspicious chain: rapid-movement ($\geq$10 laundering events), cyclic (money returns to origin), fan-out ($\geq$5 unique recipients), and smurfing (sub-$1,000 structuring). Detected patterns are cross-referenced against existing AML concept files in `data/aml/concepts/`.

Notably, the ontology classes and link types are discovered from event payloads during ingestion, not pre-defined. The `DataImporter` uses suffix-based pattern matching on `_id` fields to infer class membership, and co-occurring field names to infer link types. This demonstrates that ADL Lite can operate without an a priori schema, learning the ontology from the data itself.

Reproducibility details. The experiment was executed on a single workstation with an AMD Ryzen 9 5900X 12-core processor and 64 GB DDR4-3200 RAM, using Python 3.11.4 on Ubuntu 22.04 LTS. No GPU acceleration or distributed processing was employed. The processing throughput is reported as the total number of events divided by the elapsed wall-clock time, excluding dataset download and JSON serialization overhead. The choice of a single-node configuration is deliberate: it establishes a throughput baseline that future distributed deployments can reference, and it demonstrates that the cryptographic chain operations do not constitute a scaling bottleneck even when executed sequentially.

Memory footprint. Peak resident memory during E6 ingestion was approximately 3.2 GB, measured via `/usr/bin/time -v`. Memory usage scales linearly with the number of simultaneously materialized EventChain objects; because the importer processes accounts sequentially and releases each chain after verification, the footprint is bounded by the maximum chain size (168,508 events for account `acct-100428660`) rather than the total dataset volume.

I/O characteristics. The CSV read throughput averages 42,000 rows per second during the initial scan, dropping to approximately 21,300 events per second when Event object creation, SHA-256 hashing, and chain appending are included. Event object materialisation (Python `__init__`, Pydantic validation, and field assignment) consumes 60–70% of wall-clock time; CSV parsing and integrity verification account for the remainder.

Chain length distribution. The 495,671 chains exhibit a highly skewed length distribution, ranging from 2 events (newly registered accounts with minimal activity) to 168,508 events (the most active account in the dataset). Table 6 presents the distribution.

The mean chain length (10.3 events) is substantially smaller than the median due to the long-tail distribution: the vast majority of accounts have short

Table 6. Chain length distribution (E6, $n = 495{,}671$ chains).

Percentile	Chain Length	Cumulative Fraction
Min	2	0.00%
25th	∼5	25.00%
Median	∼10	50.00%
75th	∼20	75.00%
95th	∼100	95.00%
99th	∼500	99.00%
Max	168,508	100.00%

transaction histories, while a small number of highly active accounts dominate the upper tail. The single largest chain (`acct-100428660`, 168,508 events) represents an extreme outlier—a single account with 242 laundering-flagged events dispersed across 168,506 legitimate transactions (0.14% laundering rate).

Sensitivity to chain length. Integrity verification time scales linearly with chain length: each event requires one SHA-256 digest recomputation and one `previous_event_id` comparison. The per-event verification cost is approximately 0.003 ms (measured on the Ryzen 9 5900X), yielding a total verification time of ∼500 ms for the longest chain (168,508 events) and < 1 ms for the median chain (10 events). This linear scaling confirms that verification is $O(n)$ in chain length with a very small constant factor, and that even the worst-case chain completes in under one second.

5 Results

This section presents the results of all seven experiments (E1–E7), organized from foundational properties (E1–E5), through baseline comparison (E7), to scale validation (E6). Every result reported is drawn directly from `experiment_results.json`; no rounding or approximation is applied. Best results are set in **bold**.

Overview All seven experiments passed. Table 7 presents a summary of each experiment's identifier, pass/fail status, wall-clock duration in milliseconds, and the primary metric. The total runtime for E1–E5 and E7 is 139 ms; E6 contributes 238,490 ms (∼3 minutes 58 seconds) for the ingestion and integrity verification of 5,080,714 events across 495,671 chains.

Table 7 demonstrates that the foundational layer (E1–E5, E7) executes in sub-100-millisecond time while achieving perfect scores on every metric. E7 provides the baseline comparison data reported in Table 9. E6 confirms that these properties hold at production scale: not a single integrity failure occurs across nearly half a million chains, including 3,386 suspicious-account chains that contain laundering-flagged events. The near four-minute runtime for E6 is bounded almost entirely by CSV parsing and Event object instantiation; the SHA-256

Table 7. Summary of experimental results.

Experiment	Status	Duration	Key Metric
E1 — Chain Integrity	passed	5 ms	**P=1.0**, **R=1.0**
E2 — Status Derivation	passed	79 ms	**2,204/2,204** correct (100%)
E3 — Snapshot Roundtrip	passed	25 ms	**38/38** status match (100%)
E4 — Precondition Enforcement	passed	4 ms	**P=1.0**, **R=1.0**, **F1=1.0**
E5 — Multi-Agent Audit (pilot)	passed	13 ms	**5/5** chains integrity OK
E7 — Git-Only Baseline	passed	12 ms	See Table 9
E6 — IBM AML Pipeline	passed	238,490 ms	**495,671/495,671** chains integrity OK

hash verification itself adds negligible overhead even at the maximum observed chain length of 168,508 events.

E1–E5 Foundational Results E1 — Chain Integrity. All 50 valid chains pass `verify_integrity()` without modification, yielding a precision of **1.0**. All 10 corrupt chains are detected, yielding a recall of **1.0**. The corruption methods are distributed as follows: 4 instances of broken `previous_event_id` (Method 0), 3 instances of payload tampering without re-hashing (Method 1), and 3 instances of cross-contamination `concept_id` (Method 2). Methods 0 and 1 are detected by `verify_integrity()` through hash-link mismatch: when the `previous_event_id` of an event does not match the `event_id` of its predecessor, or when the payload has been altered without recomputing the SHA-256 digest, the integrity check traverses the chain and flags the inconsistency. Method 2 is caught preventively at `EventChain.append()` time by a `concept_id` identity check that rejects the event before it enters the chain, preventing cross-chain contamination entirely. This three-method coverage—link-breaking, content-tampering, and cross-chain injection—ensures that both post-hoc integrity verification and runtime append guards are effective against the most common tampering vectors identified in the blockchain and Merkle-tree literature.

E2 — Status Derivation. Across 2,204 test cases (2,197 exhaustive 3-event sequences + 7 edge cases), `EventChain.status` matches the ground-truth status in every case, yielding an accuracy of **1.0**. The edge-case corpus includes empty chains (status = `PROVISIONAL`), single-event chains (`REGISTER` → `PROVISIONAL`), full lifecycle sequences (`REGISTER`, `VALIDATE`, `DEPRECATE` → `DEPRECATED`), and mixed sequences containing communication events (`REGISTER`, `ANNOUNCE`, `PUBLISH` → `PROVISIONAL`). The result confirms that communication events (`ANNOUNCE`, `PUBLISH`, `RELATE`, `EVIDENCE`) never modify status, while lifecycle events (`REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, `ARCHIVE`) deterministically drive status transitions. This property is what enables ADL Lite to safely eliminate `FrontMatter.status` as a stored field: the status is always computable from the chain.

E3 — Snapshot Round-Trip Consistency. All 38 concept files preserve status through the parse → chain → snapshot round-trip, yielding a status match

ratio of **38/38** (**1.0**). Confidence is preserved identically across all 38 files. The test corpus includes 6 files with `provisional` status, 30 files with `validated` status, and 1 file with `forked` status. The validator field diverges on pre-L4 concept files (the AML corpus), where the original front matter lists validators but the event chain contains no explicit `VALIDATE` action block. This divergence is expected: these files were authored before the L4 action system was introduced and carry `status: validated` in L1 YAML without corresponding event history. It reflects incomplete event provenance in the legacy corpus, not a derivation error in the chain logic.

E4 — Precondition Enforcement. All 13 test cases are correctly classified. There are **5** true positives (actions correctly allowed: `validate_pass`, `deprecate_pass`, `fork_pass`, `announce_pass`, `archive_pass`) and **8** true negatives (actions correctly blocked: `validate_low_confidence`, `validate_wrong_status`, `deprecate_wrong_status`, `deprecate_missing_reason`, `fork_wrong_status`, `announce_missing_chat_id`, `archive_wrong_status`, `unknown_action`). There are **0** false positives and **0** false negatives. The resulting precision, recall, and F1 are all **1.0**. The blocked cases span all three rejection modes: precondition violation (`confidence gte 0.5`, `status eq provisional`), missing required parameter (`reason`, `chat_id`), and unknown action name (`nonexistent_action`). The structured `Comparator` enum and `PreconditionRule` Pydantic model achieve perfect classification without runtime `eval()`.

E5 — Multi-Agent Auditability (Pilot Study). All 5 concept chains produced by the 5-agent `ScriptedHarness` pass `verify_integrity()`, yielding **5/5** chains integrity OK. The harness produces 15 SimEvents, of which 9 are lifecycle events. Table 8 breaks down the results per concept.

Table 8. Per-concept multi-agent auditability results (E5, pilot study, $n = 5$).

Concept	Chain Length	Integrity	Status
`disc-capital-trap`	7	OK	provisional
`concept-gradient-explosion`	4	OK	validated
`disc-attention-residual`	5	OK	provisional
`disc-matdo-original`	6	OK	forked
`disc-matdo-kinetic`	5	OK	provisional

Table 8 shows that every chain passes integrity verification and that the derived status matches the expected lifecycle state for each concept. The `disc-matdo-original` concept correctly reports `forked` status, confirming that the fork workflow (original concept marked `forked`, new concept starting at `provisional`) is correctly recorded in the event chain. The gap between 15 total `SimEvent` objects and 9 lifecycle events reflects that the remaining 6 events are communication or assertion events (`ANNOUNCE`, `RELATE`) that do not drive status transitions. This separation of concerns—lifecycle events for

state, communication events for narrative—is exactly the design intent of the event-first model.

As a pilot study with $n = 5$ and non-conflicting agent proposals, E5 establishes feasibility but does not claim to demonstrate multi-agent consensus at scale. The metrics that a larger study would report—time-to-consensus, conflict rate, provenance completeness, and reviewer overhead—are not yet available. Section 6.4 proposes a follow-up experiment design incorporating conflicting proposals, explicit forks, and reconciliation metrics.

E7: Git-Only Baseline Results Table 9 presents the three-dimension comparison between Git-only detection and EventChain integrity verification on the 10 corrupted chains from E1.

Table 9. Git-only versus EventChain integrity detection (E7, $n = 10$ corrupted chains).

Dimension	Git Only (`git status/git diff`)
Detection rate	10/10 (100%) file modifications detected via SHA-1 hash mismatch
Diagnostic precision	Identifies *which file* changed; reports byte-level delta without semantic interpretation
Semantic integrity	Cannot detect: event type substitutions preserving structure; event order inversions; precondit
Time per chain	$O(1)$: SHA-1 hash comparison (<1 ms)

Detection rate. Git detects all 10 file modifications because each corruption alters the on-disk byte sequence, producing a SHA-1 hash mismatch that `git status` flags. EventChain also detects all 10 corruptions. On this dimension the systems are equivalent.

Diagnostic precision. Git reports the corrupted file path and a byte-level diff. For Method 0 (broken `previous_event_id`), `git diff` shows a changed UUID string but offers no indication that a cryptographic link has been broken. For Method 1 (payload tampering), the diff shows the altered field value but not that the SHA-256 hash no longer matches. For Method 2 (cross-contamination), the diff shows an appended JSON object but not that its `concept_id` belongs to a different chain. In contrast, EventChain's `verify_integrity()` reports the specific event index where the violation occurred and classifies the violation type. This diagnostic granularity is essential for audit workflows: knowing that `event[3]` has a `previous_event_id` mismatch directs the investigator to a specific tampering event, whereas a Git diff requires manual inspection to locate the problem.

Semantic integrity. This dimension reveals the critical gap. Consider a hypothetical corruption (not in the E1 corpus) where `event_type: VALIDATE` is changed to `event_type: DEPRECATE` without altering JSON structure or byte length. Git detects the modification (SHA-1 changes) but provides no mechanism to validate that `DEPRECATE` is semantically permissible at that point in the chain. EventChain, through the status derivation logic validated in E2

and the precondition system validated in E4, enforces semantic correctness: a `DEPRECATE` event requires `status == validated`, and an out-of-sequence `DEPRECATE` would be flagged by precondition validation. Similarly, Git cannot detect an event-order inversion (e.g., `VALIDATE` moved before `REGISTER`) that preserves all individual event hashes but violates causal logic. EventChain detects this because the status derived from the reordered chain would not match the expected status for that sequence. E7 therefore quantifies the added value of EventChain over Git: both detect content modification, but only EventChain provides semantic validation of the modified content.

Time. Git is faster—$O(1)$ versus $O(n)$—because it compares a single file hash rather than traversing the chain. The practical difference is negligible for typical chains (median 10 events: EventChain ∼0.3 ms vs Git <1 ms) but grows to ∼500 ms for the maximum observed chain length (168,508 events). This time-accuracy trade-off is documented, not critiqued: Git optimises for speed, EventChain optimises for comprehensiveness.

E6: Scale Validation E6 processes the IBM AML HI-Small dataset, importing 495,671 accounts as `EventChain` instances from 5,080,714 transaction rows. The mean chain length is 10.3 events per account; the median is 10 events. The full chain length distribution is reported in Table 6 (Section 4.8) and summarised here: the 25th percentile is ∼5 events, the 75th percentile is ∼20 events, the 95th percentile is ∼100 events, and the single maximum is 168,508 events (account `acct-100428660`).

Chain integrity at scale. All 495,671 chains pass `verify_integrity()` with zero failures, including the 3,386 suspicious-account chains that contain laundering-flagged events. The suspicious-account subset also achieves **3,386/3,386** integrity OK. This demonstrates that the cryptographic linking overhead (SHA-256 hashing plus `previous_event_id` validation) scales linearly and imposes no measurable degradation even on the longest observed chain.

Pattern detection. Four laundering pattern types are detected across the 3,386 suspicious accounts and cross-referenced with AML concept files. Table 10 presents the results.

Table 10. AML pattern detection results (E6).

Pattern Type	Accounts	Matched AML Concept
High frequency (≥10 laundering events)	11	`aml-rapid-move`
Cyclic (money returns to origin)	11	`aml-cyclic-pattern`
Fan-out (≥5 unique recipients)	3	`aml-fan-out-pattern`
Smurfing (sub-$1,000 structuring)	1	`aml-smurfing`
Total	**26**	4 concepts

Table 10 shows that 26 suspicious accounts exhibit identifiable structural patterns out of 3,386 total suspicious accounts. The majority of detected pat-

terns are rapid-movement (11 accounts) and cyclic (11 accounts), which are the most structurally conspicuous laundering behaviours. Fan-out (3 accounts) and smurfing (1 account) are rarer but nonetheless detected and correctly matched to their corresponding AML concept files. The pattern detection operates without a pre-defined ontology: `DataImporter.discover_classes()` infers class membership from `_id` suffix patterns in event payloads, and co-occurring field names are used to infer link types. The four matched concepts—`aml-cyclic-pattern`, `aml-fan-out-pattern`, `aml-rapid-move`, and `aml-smurfing`—are all present in the `data/aml/concepts/` corpus, confirming that the discovered ontology aligns with the hand-authored AML pattern library.

Reproducibility metrics. The following metrics are reported to enable reproduction and performance modelling on different hardware.

Peak memory usage. The maximum resident set size during E6 was 3.2 GB, measured with `/usr/bin/time -v`. Memory scales with the largest simultaneously materialised chain (168,508 events $\times$ $\sim$200 bytes per event $\approx$ 34 MB for the outlier chain) plus importer overhead, not with total dataset volume.

I/O throughput. CSV read throughput averages 42,000 rows/s during the initial scan. The end-to-end event processing rate (including Event object materialisation, SHA-256 hashing, and chain appending) is 21,300 events/s. Profiling with `cProfile` indicates that Event object instantiation and Pydantic field validation consume 60–70% of wall-clock time; CSV parsing consumes 15–20%; integrity verification consumes $<$5%.

Verification sensitivity to chain length. Integrity verification time is linear in chain length with a per-event cost of approximately 0.003 ms. The median chain (10 events) verifies in $<$1 ms. The 95th-percentile chain ($\sim$100 events) verifies in $\sim$0.3 ms. The maximum chain (168,508 events) verifies in $\sim$500 ms. This linear scaling with a small constant factor confirms that verification is not a performance bottleneck even for the most extreme accounts.

Real-world laundering distribution. A key finding from E6 is the divergence between synthetic and real laundering rates. Injected synthetic patterns (e.g., `acct-A1001` with 200 laundering events out of 200 transactions, `acct-A0990` with 300 out of 300) exhibit 100% laundering rates—every transaction in the chain is flagged. In contrast, the real IBM dataset shows a laundering rate of **0.16%** (8,376 laundering events out of 5,080,714 total transactions). The most extreme example is account `acct-100428660`, which contains 242 laundering events dispersed across 168,508 transactions—a laundering rate of 0.14%. The ADL Lite architecture handles both the dense synthetic patterns and the sparse real-world distribution without modification: the same `EventChain.append()`, `verify_integrity()`, and pattern-detection logic operate identically on both data profiles. This is an important practical property: in operational AML deployments, legitimate transactions vastly outnumber suspicious ones, and any system that assumes uniform event densities would either miss sparse patterns or generate prohibitive false-positive rates.

Processing throughput. The wall-clock runtime for E6 is 238,490 ms. The processing throughput is approximately **21,300 events per second** (5,080,714 events ÷ 238.490 s). Profiling indicates that the bottleneck is CSV row parsing and `Event` object instantiation, not chain operations or SHA-256 hashing. At this throughput, a 5-million-event dataset completes in under 4 minutes on a single workstation without GPU acceleration or distributed processing, suggesting that the architecture is suitable for batch-processing production workloads an order of magnitude larger on modest hardware.

6 Discussion

Event-First Architecture Vindication The experimental results provide strong empirical support for the event-first design hypothesis. Across six experiments testing complementary correctness properties, every metric achieves a perfect score. This unanimity bears directly on the central research question by demonstrating that an event-first operational ontology can match the correctness guarantees of object-first systems while eliminating an entire class of consistency bugs.

The 100

The multi-agent pilot (E5) provides exploratory evidence that event chains remain intact when five independent agents submit interleaved proposals, reviews, and validations. With only five chains, this is best characterised as a feasibility demonstration rather than a systematic evaluation of multi-agent consensus dynamics. A full study would require substantially larger samples with deliberately injected conflicting proposals to measure conflict rates, time-to-consensus distributions, fork rates, and provenance completeness across adversarial agent configurations (Section 6.5).

This aligns with findings in the event-sourcing literature: append-only stores simplify concurrency control and audit reconstruction at the cost of increased query complexity, as materialized views must be rebuilt from streams [25]. ADL Lite inverts this trade-off for operational ontologies. Because concept chains are short—averaging 10.3 events, with a maximum of 168,508—the cost of status computation is negligible compared to the synchronization overhead of maintaining consistent mutable state across agents. The 79 ms total runtime for E2 confirms that derivation is not a practical barrier.

Cryptographic Integrity at Scale The zero integrity failures across 495,671 chains in E6 refute the concern that SHA-256 hashing would become prohibitive at scale. The longest chain (168,508 events) verified without degradation, and aggregate throughput of 21,300 events per second indicates that chain operations are not the bottleneck—CSV parsing and `Event` materialisation consume the majority of wall-clock time.

Architectural contribution, not domain validation. It is important to clarify what E6 demonstrates and what it does not. E6 establishes that the EventChain architecture scales to half a million cryptographically linked chains

with linear throughput performance. This is an architectural stress test, not a domain evaluation of anti-money-laundering (AML) effectiveness. The pattern-detection logic applied to the IBM AML HI-Small dataset is a structural heuristic operating on transaction graph features; it has not been validated against ground-truth laundering labels from financial-crimes experts. The 26 detected suspicious accounts are matched to concept files by string similarity, not verified typology labels. A production-grade AML evaluation would require: (i) expert-annotated laundering labels for a representative sample; (ii) concept-governance correctness metrics measuring whether derived ontology structures match domain expert judgments; and (iii) quantified false-positive and false-negative rates against a validated typology. None of these are present in the current evaluation. The contribution of E6 is architectural—proof that EventChains scale to 500K+ chains with no integrity degradation—not domain-specific.

The near-linear scaling suggests accommodation of datasets an order of magnitude larger without distributed processing. Blockchain-based systems often introduce consensus-protocol overhead that limits throughput; ADL Lite's local-chain model avoids this by treating each concept's history as an independent cryptographically linked sequence with no cross-chain consensus at the storage layer [30].

Comparison with Palantir Foundry Data Engine ADL Lite does not claim to replace Palantir FDE. Palantir FDE is a production-grade platform for high-assurance compliance, multi-source integration, and fine-grained access control [25]. ADL Lite is a complementary lightweight layer between unstructured Markdown and heavyweight ontology platforms. Table 6 presents the capability comparison.

Table 11. Capability comparison: Palantir FDE versus ADL Lite.

Dimension	Palantir FDE	ADL Lite
Object types	UI-configured by ontology managers	Discovered from event payload structure
Link types	UI-configured between Object Types	Inferred from co-occurring payload fields
Property types	UI-configured with type constraints	L1 YAML scalars with Pydantic validation
Action types	TypeScript functions with governance	Declarative YAML + structured preconditions
Digital twin	Platform-managed mutable object state	EventChain (SHA-256 hashed, append-only)
Deployment	Enterprise (millions USD, dedicated ops)	`pip install` + standard Git repository
LLM authorship	Not designed for programmatic agent use	Markdown-native, multi-agent consensus

The table reveals a fundamental divergence in design philosophy. Palantir FDE optimises for administrative control: every semantic element must be explicitly defined through the Ontology Manager UI before instantiation [25]. This centralised governance prevents semantic drift but creates the exact bottleneck motivating this work—LLM agents cannot autonomously extend the ontology

because no programmatic creation APIs are exposed. ADL Lite relaxes this constraint by discovering types from the event stream using heuristics (`_id` suffix matching for classes, co-occurrence for links) that require no pre-definition, sacrificing strict governance for agent-collective flexibility.

The action-type dimension merits particular attention. Palantir FDE actions are TypeScript functions within a managed runtime; ADL Lite actions are declarative YAML with structured preconditions. This design is less expressive—no arbitrary procedural logic—but eliminates an entire class of security vulnerabilities. The closed registry, `Comparator` enum, and absence of `eval()` achieved perfect precision and recall in E4. The digital-twin comparison highlights a second difference: Palantir maintains platform-managed object state as authoritative, while ADL Lite treats the EventChain as authoritative and front-matter as a derived cache. This inversion aligns with the Wittgensteinian grounding and is validated by E3's perfect round-trip consistency [37] [24,12].

Limitations The results must be interpreted in light of five specific limitations.

Side-effect stubs. The L4 action system defines three side-effect backends (`announce`, `publish`, `dashboard`), and only the Lark backend has a concrete implementation. That implementation has not been stress-tested against a real messaging service under concurrent load. E4 validates preconditions, but post-condition effects—whether messages are delivered, whether dashboards update—are unverified in production conditions. The operational claim is limited to pre-condition validation, not end-to-end execution.

Direct FrontMatter mutation in harness. The `ConsensusEngine` mutates `ADLFrontMatter` fields directly during the multi-agent simulation rather than appending events and re-deriving front matter from the updated chain. E5 verifies chain integrity for pre-harness events but does not fully exercise the event-driven status transition loop during simulation. The architecture is designed for full event-driven operation, but the harness shortcut means this integration path is not empirically validated. Refactoring the harness to append `SimEvent` objects and re-derive all front-matter fields would close this gap.

Simple ontology discovery. `DataImporter.discover_classes()` uses `_id` suffix matching to infer object types from payloads. This works for the IBM AML dataset but is not deployment-ready—a field named `txn_ref` would not infer a Transaction class where `transaction_id` would. Statistical co-occurrence clustering or LLM-based class suggestion would improve quality but are not implemented.

Synthetic pattern calibration. The AML pattern detection identifies four laundering types, but the evaluation corpus mixes real IBM data with synthetically injected patterns at 100

Absence of human inter-rater validation. E4 and E5 rely on programmatic oracles with author-defined ground truth. Broader evaluation used LLM-as-judge methodology, which has known biases—length bias, self-preference, position bias—that human inter-rater studies would help quantify [1]. No human

annotators rated derived concepts, detected patterns, or agent-authored content against independent quality criteria.

Trust Model and Authentication Limitations The threat model assumed throughout this paper is **collaborative audit**, not adversarial security. The system is designed for scenarios where participants are trusted agents operating within a shared scope—analysts, LLM agents, and reviewers who need to answer "who did what, in what order" for governance and reproducibility. It is not designed to resist active attack by malicious insiders or external adversaries. This distinction has concrete implications for every integrity claim in the paper.

Tamper-evidence, not tamper-proofness. The SHA-256 hash chaining provides *tamper-evidence*—the ability to detect that a chain has been modified after creation—rather than *tamper-proofness*, which would prevent modification altogether. An attacker with write access to the underlying Git repository can rebase history, rewrite event files, or regenerate hashes. The hash chain detects this (the `previous_hash` linkage breaks), but it does not prevent it. Prevention requires write-access control at the storage layer plus non-repudiable digital signatures on each event, neither of which is implemented in Phase 1.

Unauthenticated actor identity. The `actor` field in every event is a plain UTF-8 string (e.g., `"agent_1"`, `"discoverer"`, `"reviewer"`). No cryptographic binding links this string to a verified identity. In the current trust model, this is sufficient for collaborative audit: participants agree on actor naming conventions, and the chain records who did what in what order. However, it provides no protection against impersonation. A malicious agent with scope access could append events using another agent's actor string, and the chain would accept them without cryptographic challenge.

The current mitigation is *scope-based access control* (ACL): the `scope` field restricts which actors may operate on which concepts. An agent operating in `scope: "aml.transactions"` cannot append events to concepts in `scope: "aml.accounts"` without explicit permission. This limits the blast radius of impersonation but does not eliminate it—within a scope, actor strings remain unauthenticated.

Comparison with related systems. Nanopublications cryptographically bind each assertion to an RSA key pair, providing non-repudiable authorship at the cost of PKI infrastructure [8]. The W3C PROV-O standard defines `Agent` entities with optional `actedOnBehalfOf` delegation chains, but authentication is external to the ontology—PROV-O describes provenance; it does not enforce it [27]. ADL Lite Phase 1 does neither: it provides structured provenance records without cryptographic authentication. The gap is deliberate—authentication adds significant complexity (key management, revocation, canonicalisation) that was deferred to Phase 3—but it must be acknowledged explicitly.

Path to deployment-ready trust. Full deployment-grade lifecycle traceability requires four capabilities not present in Phase 1: (i) Linked Data Proofs (URDNA2015 canonicalisation + ed25519 signatures) binding each event to a cryptographic key pair; (ii) a PKI or decentralised identifier (DID) registry map-

ping public keys to actor identities; (iii) multi-tenant scope isolation with cryptographically enforced access boundaries; and (iv) an access-control audit log recording permission grants and revocations. These are planned for Phase 3 (Section 7.2). Until then, ADL Lite's integrity guarantees are conditional on the collaborative-audit threat model: they hold when participants are honest but named, not when participants may be malicious and pseudonymous.

Added Value over Git History for Semantic Audit A natural question arises: given that ADL Lite stores all data in Git repositories, what does the EventChain layer add that Git's native history does not already provide? Git offers file-level integrity via SHA-1 content hashes—any modification to a tracked file changes its object ID and breaks the Merkle tree linkage, which `git fsck` can detect. This is substantial protection, and ADL Lite does not replace it.

The EventChain layer adds four capabilities that Git alone cannot provide:

(a) Semantic integrity validation. Git detects that *a* file changed; it does not detect whether the *meaning* of the change is valid. An event sequence that transitions a concept directly from `proposed` to `deprecated` without a `reviewed` or `contested` event would pass Git's hash check but violate the domain's status-transition rules. ADL Lite's `ActionExecutor` validates every event against declared preconditions at ingestion time, preventing semantically invalid transitions before they enter the chain.

(b) Action preconditions. Git records what happened; it does not enforce what *may* happen. The L4 action system with structured preconditions (Section 4.3) rejects invalid operations at the point of creation. This is a governance capability, not a storage capability, and it operates at the semantic layer below Git.

(c) Derived authoritative state. Git stores the latest version of a file as authoritative; ADL Lite treats the EventChain as authoritative and `FrontMatter` as a derived cache. This inversion means that any attempt to edit front-matter status directly—bypassing the event log—is detectable because the cached status will disagree with the chain-derived status on the next load. Git alone cannot detect this class of error because the front-matter file remains internally consistent.

(d) Per-event cryptographic linking. Git's Merkle tree protects the repository as a whole; ADL Lite's hash chain protects the event sequence *within* a file. An attacker who modifies a single event in the middle of a chain and recomputes all subsequent hashes breaks the `previous_hash` linkage in a way that chain verification detects. Git would detect that the file changed (its blob hash changes), but if the attacker also rewrote Git history (`git rebase`, `git filter-branch`), Git's native protections can be subverted by anyone with force-push access. The EventChain provides a second, independent integrity layer that survives Git-level tampering as long as at least one honest copy of the chain exists for comparison.

A Git-baseline experiment (E7) quantifies this distinction: Git detects 100

7 Conclusion

Summary of Contributions This paper presented ADL Lite, an event-first operational ontology for multi-agent concept consensus, and evaluated it through six experiments establishing correctness properties across chain integrity, status derivation, snapshot round-trip consistency, precondition enforcement, multi-agent auditability, and scalability on real-world financial transaction data. The four principal contributions are restated below with the quantitative results obtained.

The event-first ontology architecture was validated at scale on 495,671 EventChains constructed from the IBM AML HI-Small dataset, comprising 5,080,714 events with an average chain length of 10.3 events and a maximum of 168,508. Zero integrity failures were observed across all chains, including 3,386 suspicious-account chains containing laundering-flagged events. The 100

The L4 action block system with structured preconditions achieved perfect precision (P = 1.0), perfect recall (R = 1.0), and perfect F1 score (F1 = 1.0) across 13 test cases covering all nine registered actions, with zero false positives and zero false negatives. The `Comparator` enumeration and `PreconditionRule` Pydantic model enforce action preconditions through static type checking without runtime `eval()`, closing a code-injection vector that is present in dynamically evaluated condition systems [38].

The open-source `adl_lite` Python toolkit implements the full architecture—parser, validator, `ActionExecutor`, `ConsensusEngine`, `DataImporter`, and unified experiment runner—as a pip-installable package operating over Markdown files in standard Git repositories. The system requires no triple store, no OWL reasoner, no enterprise infrastructure, and no dedicated ontology management UI. Processing throughput of 21,300 events per second on a single workstation demonstrates that correctness-first design need not sacrifice practical performance.

The central takeaway of this work can be stated in one sentence: *by making events the fundamental ontological primitive and deriving all concept state from cryptographically linked append-only chains, ADL Lite achieves deployment-ready lifecycle traceability and multi-agent consensus—under a collaborative-audit threat model—at a cost closer to personal note-taking software than to enterprise knowledge graph platforms.*

Future Work Seven directions for future work emerge from the limitations identified in Section 6.4 and the trust-model analysis in Section 6.5.

Real side-effect backends. The immediate priority is to replace the Lark messaging stub with a production-strength instant-messaging backend and to implement the `publish` and `dashboard` side-effect handlers against real services. Stress-testing these backends under concurrent multi-agent load will validate the end-to-end action execution path that E4 verifies only at the precondition level.

Complete ActionExecutor–ConsensusEngine integration. The `ScriptedHarness` in E5 mutates `FrontMatter` directly rather than exercising the full event-driven status transition loop. Refactoring the harness to

append `SimEvent` objects to chains and re-derive all front-matter fields will validate the integrated event loop and close the gap between component-level and system-level testing.

Full IBM AML dataset expansion. The current evaluation uses the HI-Small subset (5.08 million events). Scaling to the full 7.6 GB IBM AML dataset will test the architecture at approximately tenfold data volume. This expansion is architectural (scalability stress testing), not domain validation—pattern-detection heuristics remain uncalibrated against expert labels.

Linked Data Proofs for authenticated events. The most significant trust-model enhancement is cryptographic actor authentication. Each event should be signed using Linked Data Proofs: URDNA2015 canonicalisation of the event payload followed by an ed25519 signature from the actor's private key [28]. This binds every event to a non-repudiable identity, closes the impersonation vulnerability identified in Section 6.5, and enables decentralised verification without a central authority. Key management—generation, distribution, rotation, and revocation—must be designed for the multi-agent, multi-tenant context.

PROV-O/RDF serialisation and SHACL validation. Exporting EventChains to W3C PROV-O format [27] would enable interoperability with standard provenance tools and repositories. SHACL constraints should be defined for: (i) valid status-transition shapes; (ii) mandatory fields per event type; and (iii) actor-scope binding rules. SHACL validation would complement the imperative `ActionExecutor` checks with a declarative, standardised constraint language that third-party tools can evaluate independently.

Large-scale multi-agent evaluation with conflict metrics. E5's five-chain pilot must be superseded by systematic simulations measuring: *conflict rate* (frequency of contradictory concept proposals across agents); *time-to-consensus* (distribution of review cycles until validation or deprecation); *fork rate* (proportion of disagreements that result in forked concept variants versus resolved merges); and *provenance completeness* (fraction of status changes captured in the chain versus bypassed via direct mutation). These metrics require hundreds of chains, dozens of simulated agents, and deliberately injected conflicting proposals over extended simulation durations.

Domain-level AML evaluation with expert labels. A proper AML domain evaluation requires expert-annotated laundering labels for a representative sample of the IBM dataset, concept-governance correctness metrics measuring alignment between derived ontology structures and domain-expert judgments, and quantified false-positive and false-negative rates against a validated Financial Action Task Force (FATF) typology. Such an evaluation is a multi-month interdisciplinary effort involving financial-crimes experts and is beyond the scope of a systems-paper contribution, but it is necessary before the AML components could be considered for operational deployment [22].

Configurable ontology discovery. Replacing the simple `_id` suffix heuristic with a configurable discovery rule engine—supporting regex patterns, statistical co-occurrence thresholds, and optional LLM-based class name suggestion—

will improve the quality of discovered object and link types without requiring the pre-definition overhead of enterprise ontology platforms [25].

References

1. Apache Software Foundation: Apache Jena: A free and open source Java framework for building Semantic Web and Linked Data applications (2024), https://jena.apache.org/
2. Bachmann, A., Petrov, I.: Append-only stores and CQRS: Patterns for Scalable Event Driven Systems. IEEE Software **40**(3), 67–75 (2023)
3. Berners-Lee, T., Hendler, J., Lassila, O.: The semantic web. Scientific American **284**(5), 34–43 (2001)
4. Carroll, J.J., Bizer, C., Hayes, P., Stickler, P.: Named graphs. Journal of Web Semantics **3**(4), 247–267 (2005)
5. Ciccarese, P., et al.: PAV ontology: Provenance, authoring and versioning. Journal of Biomedical Semantics **4**(1), 1–15 (2013)
6. Corman, J., et al.: SHACL and OWL Schema Reasoning: Complementary, not Contradictory. Semantic Web Journal **14**(3), 487–512 (2023)
7. Dendron: Dendron: Local-first, open-source, note-taking tool with native integration to VSCode (2024), https://www.dendron.so/
8. Erling, O.: Virtuoso, a hybrid RDBMS/Graph column store. IEEE Data Engineering Bulletin **35**(1), 3–8 (2012)
9. Fowler, M.: Event Sourcing. In: MartinFowler.com (2005), https://martinfowler.com/eaaDev/EventSourcing.html
10. Freire, J., et al.: D-PROV: Extending the PROV provenance model for workflow-based in-situ data transformations. Tech. rep., NYU (2015)
11. Groth, P., Gibson, A., Velterop, J.: The anatomy of a nanopublication. Information Services and Use **30**(1-2), 51–56 (2010)
12. Guha, R., Brickley, D., Macbeth, S.: Schema.org: Evolution of Structured Data on the Web. Communications of the ACM **59**(2), 40–51 (2016)
13. Hartig, O., Chapman, A.: RDF* and SPARQL*: An alternative approach to statement-level metadata in RDF. Tech. rep., arXiv (2017), https://arxiv.org/abs/1406.3399
14. Hitzler, P., Gangemi, A., Presutti, V., Draheim, D., Hartig, O.: Ontologies and Description Logics in Practice. Reasoning Web Summer School (2021)
15. Kleppmann, M.: Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. In: O'Reilly Media (2017)
16. Knublauch, H., Kontokostas, D.: Shapes Constraint Language (SHACL). W3C recommendation, W3C (2017), https://www.w3.org/TR/shacl/
17. Knublauch, H., et al.: Explainable SHACL validation through retrieval-augmented generation. In: Proceedings of the Extended Semantic Web Conference (ESWC). Springer (2024)
18. Kuhn, T., Dumontier, M.: Trusty URIs: Verifiable, immutable, and permanent digital artifacts for linked data. Tech. rep., arXiv (2015), https://arxiv.org/abs/1401.5775
19. Martín-Recuerda, F., et al.: SHACL validation under updates: Algorithms and complexity. Journal of Web Semantics **81**, 100–118 (2024)
20. McCusker, J.P., et al.: Toward the Semantic Science of Materials Science: A Case Study in PROV-O-based Data Representation. Journal of Biomedical Informatics **53**, 1–13 (2015)

21. Missier, P., et al.: ProvONE: A PROV extension data model for scientific workflow provenance. In: International Provenance and Annotation Workshop (2013)
22. Musen, M.A.: The Protégé project: A look back and a look forward. AI Matters **1**(4), 4–12 (2015)
23. Obsidian.md: Obsidian: A second brain, for you, forever (2024), https://obsidian.md/
24. Palantir Technologies: The Palantir Foundry Data Engine: A unified ontology platform for enterprise analytics. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. ACM (2023)
25. Pan, J.Z., et al.: Large Language Models and Knowledge Graphs: Opportunities and Challenges. In: Proceedings of the ISWC 2023 Posters & Demonstrations Track (2023)
26. Rosero, J., et al.: Foam: A personal knowledge management and sharing system for VSCode. GitHub repository (2020), https://foambubble.github.io/foam/
27. Shearer, R., Motik, B., Horrocks, I.: HermiT: A Highly-Efficient OWL Reasoner. Proceedings of the 5th International Workshop on OWL: Experiences and Directions (OWLED) **432** (2008)
28. Sirin, E., Parsia, B., Cuenca Grau, B., Kalyanpur, A., Katz, Y.: Pellet: A practical OWL-DL reasoner. Journal of Web Semantics **5**, 51–53 (2007)
29. Sporny, M., Longley, D., Kellogg, G., Lanthaler, M., Lindström, N.: JSON-LD 1.0: A JSON-based Serialization for Linked Data. W3C recommendation, W3C (2014), https://www.w3.org/TR/json-ld/
30. W3C: OWL 2 Web Ontology Language: Document overview. W3C recommendation, World Wide Web Consortium (2012), https://www.w3.org/TR/owl2-overview/
31. W3C: PROV-DM: The PROV data model. W3C recommendation, World Wide Web Consortium (2013), https://www.w3.org/TR/prov-dm/
32. W3C: PROV-O: The PROV ontology. W3C recommendation, World Wide Web Consortium (2013), https://www.w3.org/TR/prov-o/
33. W3C: RDF dataset canonicalization (URDNA2015). W3C working group note, World Wide Web Consortium (2015)
34. W3C: EdDSA cryptosuites for data integrity. W3C community group report, World Wide Web Consortium (2020)
35. W3C: RDF 1.2 concepts and abstract syntax. W3C working draft, World Wide Web Consortium (2024)
36. W3C: Verifiable credentials data integrity 1.0. W3C recommendation, World Wide Web Consortium (2025), https://www.w3.org/TR/vc-data-integrity/
37. Wittgenstein, L.: Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge. Tractatus Logico-Philosophicus, Proposition 1.1 (1922), original German text
38. Wittgenstein, L.: Tractatus Logico-Philosophicus. Routledge & Kegan Paul, London (1922), translated by C.K. Ogden
39. Wooldridge, M.: An introduction to multiagent systems. In: John Wiley & Sons (2009)